

Insights of Particle Image Velocimetry

JF Krawczynski

2026-09-23

Table of contents

About	4
Assessment	5
Schedule	6
Recommended Readings and Media	7
Theory	8
Basics of PIV	9
How to estimate a velocity-field from a pair of grayscale particle images?	10
A slight variant	16
OpenPIV tutorial	20
Processing	21
Post-processing	21
Save and plot the results	22
Another example	23
Advanced PIV techniques	26
How window deformation works	26
How multi-pass processing works	26
Vector Validation Techniques	26
Vector Replacement	27
Configuring Advanced PIV Analysis	27
Key Settings	27
Example Usage	27
Summary	32
Practice	33
Measurements	34
Experimental setup	34
Seeding particles	34
Light source	34
Image recording	34

Acquisition protocol	35
Before your slot: target realizations per series	35
One-time setup	35
Per case (flow $\times$ regime)	35
Summary	35
Appendix: converting your images	36
Processing	39
Dataset	40
Image-quality diagnostics	42
Masking	43
Batch multi-pass processing	44
Quick quality control	46
Summary & next step	49
Analysis	50
1. Loading a batch into 2D fields	51
2. Vorticity	53
3. Case-specific diagnostics	54
3.1 Cylinder: wake velocity-deficit profile	54
3.2 Step: recirculation length	55
4. Vortex shedding & Strouhal number	57
5. Regime & flow comparison	60
6. Discussion — for your report	61
References	63

About

This site hosts the course materials for fluid mechanics measurements by PIV for the course unit **UM5MEE12 - Experimental Methods and Data Processing** taught in the [Master of Engineering Sciences](#) at the [Faculty of Engineering](#) of [Sorbonne University](#). The aim is to understand the basic concepts inherent to measuring velocity fields in fluid flows. The slides of the introductory lecture can be found [here](#).

Note

This course is partly adapted from the [documentation](#) of [OpenPIV](#), licensed under the GNU General Public License v2.0.

The course content is divided into two parts:

- The theoretical aspects of the course, including:
 - the fundamental physical considerations related to the tracer particles used to measure the flow
 - the algorithms developed to obtain the velocity vector field from the analysis of particle images.
- The practical aspects:
 - a numerical part to better understand the evaluation methods using PIV. The images processed will be generated synthetically
 - an experimental part to understand the trade-offs required to obtain high-quality images for processing.

In this course, we will mainly use the following tools, with which students should be familiar:

- [Python](#)
- [Numpy](#)
- [Matplotlib](#)

Tip

A **PDF version** of this website is available for offline viewing (please avoid printing!).

Assessment

Assessment will be based on a short lab report (for the numerical part) and a project report focusing on the analysis of experimental data obtained during the lab sessions.

Schedule

Here is the planned organization of the sessions:

#	Date	Topic
1	09/23/2026	Introduction to velocity field measurement by PIV, Lecture 1
2	09/30/2026	Algorithms and experimental specifics, Lab 1 + Lab 2
3	10/07/2026	Algorithms and experimental specifics, Lab 1 + Lab 2

Recommended Readings and Media

Below is a list of useful resources for your project, organized by topic.

Python for Scientific Programming

- [Un Zeste de Python](#)
- [Les bases de numpy et matplotlib](#)
- [From Python to Numpy](#)
- [Scientific Visualization: Python + Matplotlib](#) (N. P. Rougier 2021)

Scientific outputs: figures and reports

- [Ten Simple Rules for Better Figures](#) (M. D. Rougier Nicolas P. 2014)
- [Wikibook LaTeX](#)
- [Typst](#)

Theory

Basics of PIV

“Particle” Image Velocimetry (PIV) is a non-intrusive state-of-the-art technique (Adrian 1991), (Raffel 2007) to get the velocity field of the flow being studied from images of small particles, called tracers.

It is based on image recording of the illuminated flow field using seeding particles (called tracers) which, when sufficiently small compared to the smallest characteristic length scales of the flow, are assumed to faithfully follow the flow dynamics (the degree to which the particles faithfully follow the flow is represented by the Stokes number). In practice, common sizes of the tracer particles are in the order of 5-100 microns. The entrained particles are generally made visible in a cross-section of the flow being studied by forming a coherent light sheet. In practice, the flow is illuminated twice using a laser light sheet, forming a plane where a camera is focused. The time delay between the pulses depends on the mean velocity and the image magnification. It is assumed that the tracer particles follow the local flow velocity between the two consecutive illuminations. The light scattered from the tracer particles is then imaged via an optical lens on a digital camera. The displacement of the particle images between two consecutive light pulses (respectively, frames) is determined through evaluation of the spatial cross-correlation function and image processing tools as implemented in OpenPIV.

The effectiveness of the measurement results strongly depends on a large number of parameters such as particles concentration, size distribution and shape, illumination source, recording device, and synchronization between the illumination, acquisition and recording systems (Huang 1997). An appropriate choice of the different parameters of the cross correlation analysis (e.g., interrogation area, time between pulses, scaling) will influence the results accuracy.

Using **simple mathematical knowledge** and **existing algorithms** written with Python, Numpy, Scipy, we will introduce the basics of PIV.

Note

These notebooks require the [OpenPIV](#) package (Python 3.10). If it isn't already installed in your environment, run `pip install openpiv` before continuing — every notebook of this course imports it directly (e.g. `from openpiv import tools, pyprocess, validation, filters, scaling`).

All the images used can be found in the [images](#) directory of the repository.

How to estimate a velocity-field from a pair of grayscale particle images?

This tutorial will follow the simplest analysis path from a pair of images to the velocity field and some post-analysis. It is a simple refresher from the [Lecture session](#).

```
# import the standard numerical and plotting packages
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
# import what is necessary from OpenPIV
from openpiv import tools, pyprocess, validation, filters, scaling
```

We have downloaded sample images from a standard PIV images project, see [pivchallenge](#)

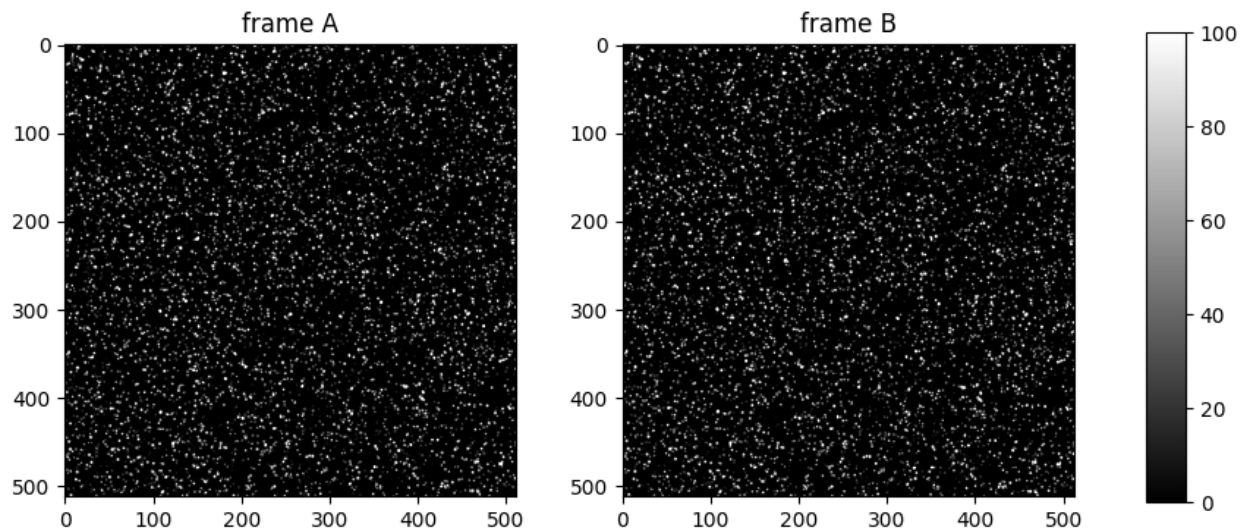
```
# load the images
a = tools.imread("./images/B005_1.tif")
b = tools.imread("./images/B005_2.tif")

fig, axs = plt.subplots(1, 2, figsize=(9, 4))

img = axs[0].imshow(a, vmin=0, vmax=100, cmap=plt.cm.gray)
axs[0].set_title('frame A')

img = axs[1].imshow(b, vmin=0, vmax=100, cmap=plt.cm.gray)
axs[1].set_title('frame B')

cbar = fig.add_axes([0.95, 0.1, 0.03, 0.8])
fig.colorbar(img, cax=cbar)
plt.show()
```



The two images show the particles at two different times. But these images are way too big and contain too many particles to manually track the movement of individual particles from one frame to the other. Instead, we can analyze small regions of interest, called interrogation windows (IW). Typically, we can start with a size of 32 x 32 pixels or smaller. Until recently, the fast algorithms used powers of 2, so the historical sizes are always powers of 2: 8, 16, 32, 64, 128, ...

Let's take the first top left windows from each image.

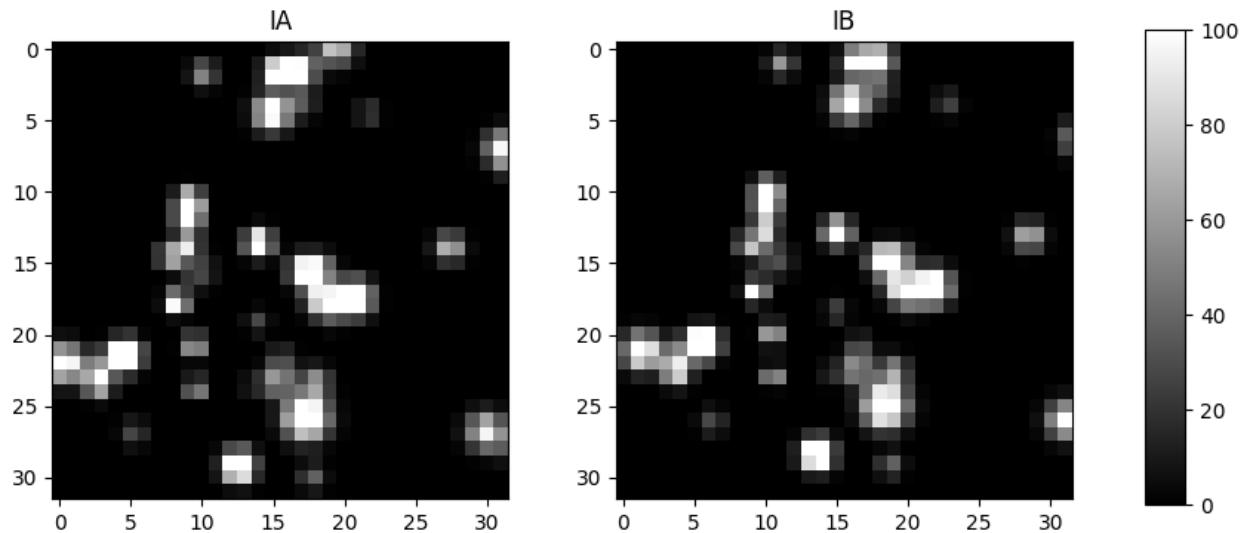
```
win_size = 32

a_win = a[:win_size, :win_size].copy()
b_win = b[:win_size, :win_size].copy()

fig, axs = plt.subplots(1, 2, figsize=(9, 4))
img = axs[0].imshow(a_win, vmin=0, vmax=100, cmap=plt.cm.gray)
axs[0].set_title('IA')

img = axs[1].imshow(b_win, vmin=0, vmax=100, cmap=plt.cm.gray)
axs[1].set_title('IB')

cbar = fig.add_axes([0.95, 0.1, 0.03, 0.8])
fig.colorbar(img, cax=cbar)
plt.show()
```



Again, if we try to manually track the movement of individual particles from frame A to frame B, it would be extremely time-consuming, especially for large image regions containing many particles. Manually identifying and matching every single particle would quickly become tedious and impractical for even a coarse velocity field estimation. That's why automated methods which can efficiently analyze the displacement of all particles within interrogation windows are preferred.

Then, let's try to find the best shift algorithmically using the principles of least squares: shift and calculate the sum of squared differences, the minimum of which should be the best shift.

```
def match_template(img, template, maxroll=8):
    # img: input image to compare
    # template: image to match against
    # maxroll: max nbr of pxl to shift in each dir. Default value is 8.
    best_dist = np.inf
    best_shift = (-1, -1)
    for y in range(-maxroll, maxroll):
        for x in range(-maxroll, maxroll):
            # calculate Euclidean distance
            dist = np.sqrt(np.sum((img - np.roll(template, (y, x), axis=(0, 1))) ** 2))
            if dist < best_dist:
                best_dist = dist
                best_shift = (y, x)
    return (best_dist, best_shift)
```

Let's check that it works by manually shifting the same image (IA):

```
match_template(np.roll(a_win, (2, 0), axis=(0, 1)), a_win)
```

```
(0.0, (2, 0))
```

Indeed, when we find the correct shift, we get zero distance. It's not so in real images:

```
match_template(b_win, a_win)
```

```
(123.80226169178009, (-1, 1))
```

Well, this is not the true displacement, but it gives a hint.

We could draw this as a vector of velocity

$$u = \frac{\Delta x \text{ pixels}}{\Delta t}, \quad v = \frac{\Delta y \text{ pixels}}{\Delta t}$$

where Δt is the time interval (delay) between the two images (or two laser pulses).

The problem is that shifting each image and repeating the loop many times is impractical:

- For a single 32×32 window, `maxroll=8` already means testing $16 \times 16 = 256$ candidate shifts, each one revisiting all 1024 pixels of the window — about 800000 operations just to locate one vector, and only to the nearest *pixel*.
- The cost grows as $O(\text{maxroll}^2)$: measured on these images, doubling `maxroll` ($4 \rightarrow 8 \rightarrow 16 \rightarrow 32$) roughly **quadruples** the time per window. A faster flow, or a larger Δt , needs a wider search range — and the cost blows up faster than that range does.

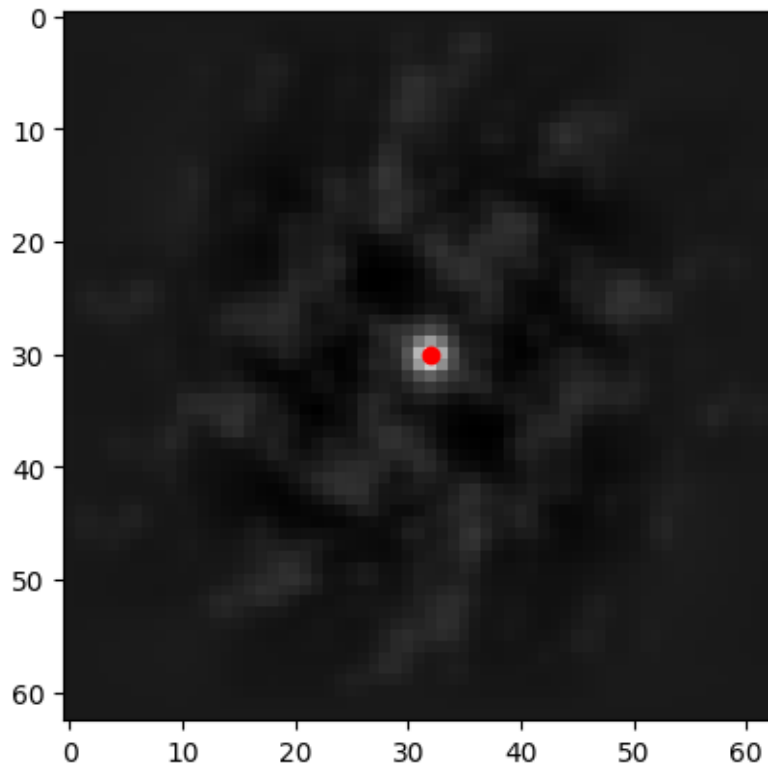
However, one can get it by using a different matching principle, based on the property called cross-correlation (cross because we use two different images). This is an efficient computational algorithm to find out the right shift: instead of testing shifts one at a time, it evaluates the match quality at *every* possible shift simultaneously via the Fourier transform — which is why its cost barely depends on the search range. You can see more details here: <http://paulbourke.net/miscellaneous/correlate/>.

```
from scipy.signal import correlate

cross_corr = correlate(b_win - b_win.mean(), a_win - a_win.mean(), method="fft")
# Note that it's approximately twice as large as the original windows, as we
# can shift a_win by a maximum of its size horizontally and vertically
# while still maintaining some overlap between the two windows.
print("Size of the correlation map: %d x %d" % cross_corr.shape)

# let's see what the cross-correlation looks like
plt.imshow(cross_corr, cmap=plt.cm.gray)
y, x = np.unravel_index(cross_corr.argmax(), cross_corr.shape)
print(f"{y=}, {x=}")
plt.plot(x, y, "ro")
plt.show()
```

Size of the correlation map: 63 x 63
y=30, x=32



The image of the correlation map shows the same result that we got manually looping. We need to subtract the center of symmetry (31, 31) to get the estimated displacement.

```
dy, dx = y - 31, x - 31
print(f"{dy=}, {dx=}")
```

dy=-1, dx=1

We can get the first velocity field by repeating this analysis for all small windows. Let's take 32 x 32 pixels windows from each image and do the loop:

```
def vel_field(curr_frame, next_frame, win_size):
    ys = np.arange(0, curr_frame.shape[0], win_size)
    xs = np.arange(0, curr_frame.shape[1], win_size)
    dys = np.zeros((len(ys), len(xs)))
    dxs = np.zeros((len(ys), len(xs)))
    for iy, y in enumerate(ys):
        for ix, x in enumerate(xs):
```

```

int_win = curr_frame[y : y + win_size, x : x + win_size]
search_win = next_frame[y : y + win_size, x : x + win_size]
cross_corr = correlate(
    search_win - search_win.mean(), int_win - int_win.mean(), method="fft"
)
dys[iy, ix], dxs[iy, ix] = (
    np.unravel_index(np.argmax(cross_corr), cross_corr.shape)
    - np.array([win_size, win_size])
    + 1
)
# draw velocity vectors from the center of each window
ys = ys + win_size / 2
xs = xs + win_size / 2
return xs, ys, dxs, dys

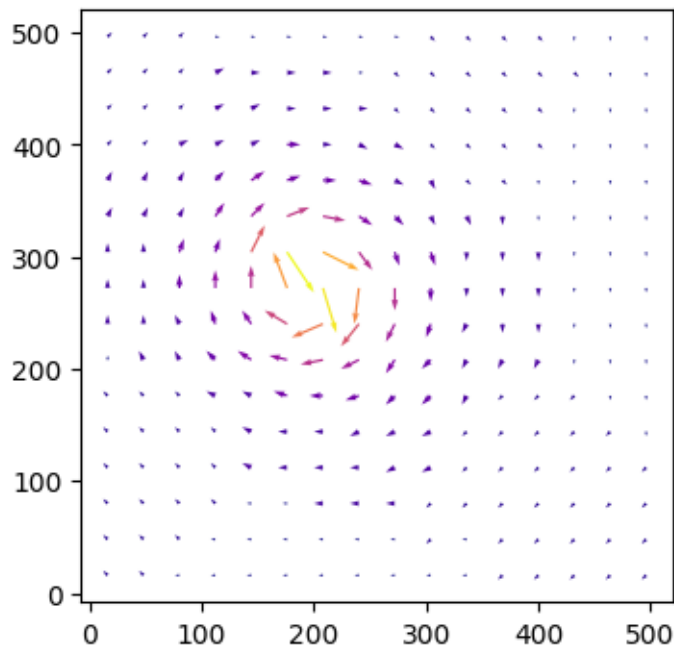
```

```

xs, ys, dxs, dys = vel_field(a, b, 32)
norm_drs = np.sqrt(dxs ** 2 + dys ** 2)

fig, ax = plt.subplots(figsize=(9, 4))
# we need these flips on y since quiver uses a bottom-left origin, while our
# arrays use a top-right origin
ax.quiver(
    xs,
    ys[::-1],
    dxs,
    -dys,
    norm_drs,
    cmap="plasma",
    angles="xy",
    scale_units="xy",
    scale=0.25,
)
ax.set_aspect("equal")
plt.show()

```



If you've followed along this far, great! Now you understand the basics.

A slight variant

We can also try to use a search window larger than the interrogation window instead of relying on zero-padding. By avoiding using zero-padding around the search window, movement detection should theoretically be a bit better, assuming that the window sizes are chosen well.

```
def vel_field_asymmetric_wins(
    curr_frame, next_frame, half_int_win_size, half_search_win_size
):
    ys = np.arange(half_int_win_size[0], curr_frame.shape[0], 2 * half_int_win_size[0])
    xs = np.arange(half_int_win_size[1], curr_frame.shape[1], 2 * half_int_win_size[1])
    dys = np.zeros((len(ys), len(xs)))
    dxs = np.zeros((len(ys), len(xs)))
    for iy, y in enumerate(ys):
        for ix, x in enumerate(xs):
            int_win = curr_frame[
                y - half_int_win_size[0] : y + half_int_win_size[0],
                x - half_int_win_size[1] : x + half_int_win_size[1],
            ]
            search_win_y_min = y - half_search_win_size[0]
            search_win_y_max = y + half_search_win_size[0]
            search_win_x_min = x - half_search_win_size[1]
            search_win_x_max = x + half_search_win_size[1]
```

```

truncated_search_win = next_frame[
    max(0, search_win_y_min) : min(b.shape[0], search_win_y_max),
    max(0, search_win_x_min) : min(b.shape[1], search_win_x_max),
]
cross_corr = correlate(
    truncated_search_win - np.mean(truncated_search_win),
    int_win - np.mean(int_win),
    mode="valid",
    method="fft",
)
dy, dx = np.unravel_index(np.argmax(cross_corr), cross_corr.shape)
# if the top of the search window got truncated, shift the origin
# up to the top edge of the (non-truncated) search window
if search_win_y_min < 0:
    dy += -search_win_y_min
# if the left of the search window got truncated, shift the origin
# over to the left edge of the (non-truncated) search window
if search_win_x_min < 0:
    dx += -search_win_x_min
# shift origin to the center of the search window
dy -= half_search_win_size[0] - half_int_win_size[0]
dx -= half_search_win_size[1] - half_int_win_size[1]
dys[iy, ix] = dy
dxs[iy, ix] = dx
return xs, ys, dxs, dys

```

```

int_win_size = np.array([32, 32])
print(f"{int_win_size=}")
assert np.all(np.array(a.shape) % int_win_size == 0)
assert np.all(int_win_size % 2 == 0)
half_int_win_size = int_win_size // 2

search_win_size = int_win_size * 2
print(f"{search_win_size=}")
assert np.all(search_win_size % 2 == 0)
half_search_win_size = search_win_size // 2
assert np.all(search_win_size > int_win_size)
print(
    "max velocity that can be detected with these window sizes: "
    + f"{half_search_win_size - half_int_win_size}"
)

```

```

int_win_size=array([32, 32])
search_win_size=array([64, 64])
max velocity that can be detected with these window sizes: [16 16]

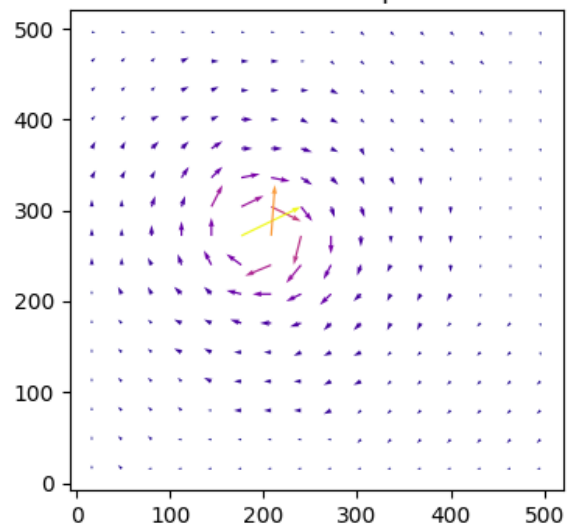
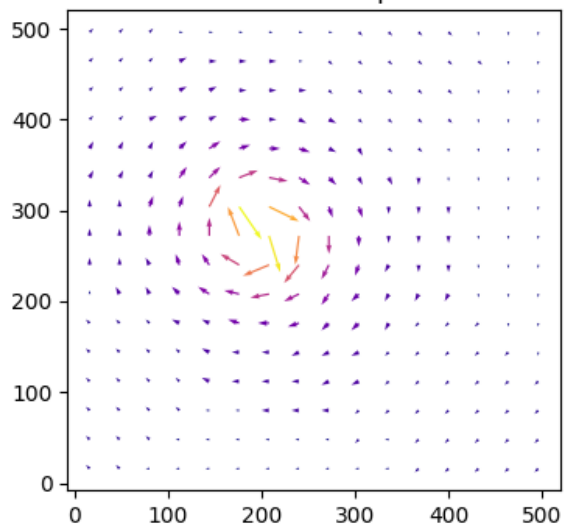
```

Making the search window larger compared to the interrogation window would allow for larger velocities to be detected.

```
xs_asym, ys_asym, dxs_asym, dys_asym = vel_field_asymmetric_wins(
    a, b, half_int_win_size, half_search_win_size
)
norm_drs_asym = np.sqrt(dxs_asym ** 2 + dys_asym ** 2)

fig, axs = plt.subplots(1, 2, figsize=(9, 4))
axs[0].quiver(
    xs,
    ys[:-1],
    dxs,
    -dys,
    norm_drs,
    cmap="plasma",
    angles="xy",
    scale_units="xy",
    scale=0.25,
)
axs[1].quiver(
    xs_asym,
    ys_asym[:-1],
    dxs_asym,
    -dys_asym,
    norm_drs_asym,
    cmap="plasma",
    angles="xy",
    scale_units="xy",
    scale=0.25,
)
axs[0].set_title(
    f"{win_size} x {win_size} int. win. + "
    f"{win_size} x {win_size} 0-padded search win."
)
axs[1].set_title(
    f"{int_win_size[0]} x {int_win_size[1]} int. win. + "
    f"{search_win_size[0]} x {search_win_size[0]} unpadded search win."
)
ax.set_aspect("equal")
plt.show()
```

32 x 32 int. win. + 32 x 32 0-padded search win. 32 x 32 int. win. + 64 x 64 unpadded search win.



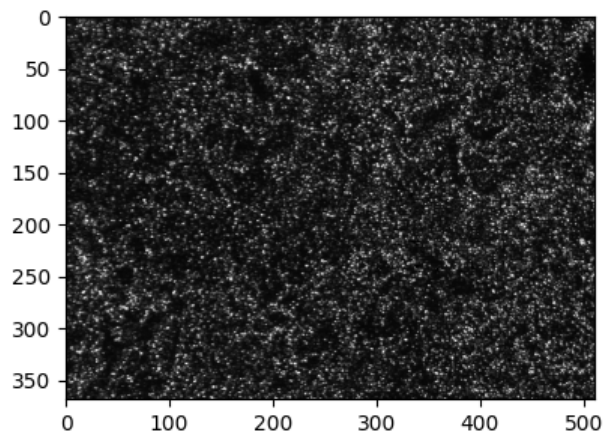
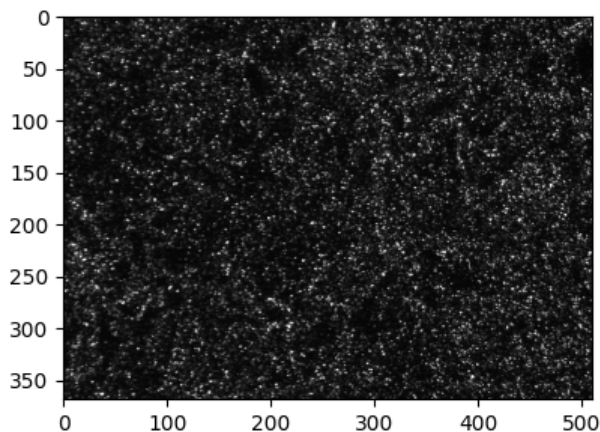
OpenPIV tutorial

This tutorial follows the simplest analysis path from the two images to the velocity field and dives deeper into the OpenPIV package functionalities.

```
# import the standard numerical and plotting packages
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
# import what is necessary from OpenPIV
from openpiv import tools, pyprocess, validation, filters, scaling
```

```
# read a pair of PIV images
frame_a = tools.imread( './images/exp1_001_b.bmp' )
frame_b = tools.imread( './images/exp1_001_c.bmp' )
```

```
# let's visualize them using matplotlib
fig,ax = plt.subplots(1,2,figsize=(10,8))
ax[0].imshow(frame_a,cmap=plt.cm.gray)
ax[1].imshow(frame_b,cmap=plt.cm.gray)
plt.show()
```



Processing

We are going to use the `extended_search_area_piv` function, which is a standard PIV cross-correlation algorithm.

This function allows the search area (`search_area_size`) in the second frame to be larger than the interrogation window in the first frame (`window_size`). Also, the search areas can overlap (`overlap`).

The `extended_search_area_piv` function will return three arrays: 1. The `u` component of the velocity field 2. The `v` component of the velocity field 3. The signal-to-noise ratio (`sig2noise`) of the cross-correlation map of each vector. The higher the signal-to-noise ratio, the higher the probability that its magnitude and direction are correct.

```
# define the PIV analysis parameters
winsize = 24 # size of the interrogation window in frame A, in pixels
searchsize = 32 # size of the window in frame B (searchsize is larger or equal to winsize), in pixels
overlap = 12 # overlap between the neighbouring windows, in pixels
dt = 0.02 # time interval of the PIV recording, in sec
```

```
u, v, sig2noise = pyprocess.extended_search_area_piv(
    frame_a.astype(np.int32),
    frame_b.astype(np.int32),
    window_size=winsize,
    overlap=overlap,
    dt=dt,
    search_area_size=searchsize,
    sig2noise_method='peak2peak',
)
```

The function `get_coordinates` finds the center of each interrogation window. This will be useful later on when plotting the vector field.

```
# get a list of coordinates for the vector field
x, y = pyprocess.get_coordinates(
    image_size=frame_a.shape,
    search_area_size=searchsize,
    overlap=overlap,
)
```

Post-processing

Strictly speaking, we are ready to plot the vector field. However, some spurious vectors might locally impact the quality of the results. It might therefore be useful to apply some filtering.

To start, let's use the function `sig2noise_val` to get a mask indicating which vectors have a minimum amount of signal-to-noise ratio. Vectors below a certain threshold are substituted by NaN. If you are not sure about which threshold value to use, try taking a look at the signal-to-noise ratio histogram with: `plt.hist(sig2noise.flatten())`.

```
# clean the peaks that are below a quality threshold
invalid_mask = validation.sig2noise_val(
    sig2noise,
    threshold = 1.3,
)
```

Another useful function is `replace_outliers`, which will find outlier vectors and substitute them with an average of neighboring vectors. The larger the `kernel_size`, the larger the considered neighborhood. This function uses an iterative image inpainting algorithm. The number of iterations can be chosen via `max_iter`.

```
# replace those that are masked as bad vectors with local interpolation
u, v = filters.replace_outliers(
    u, v,
    invalid_mask,
    method='localmean',
    max_iter=3,
    kernel_size=3,
)
```

Next, we are going to convert pixels to millimeters.

```
# scale the results from pix/dt to mm/sec
x, y, u, v = scaling.uniform(x, y, u, v, scaling_factor = 96.52 ) # 96.52 pixels/millimeter
```

Save and plot the results

The function `save` is used to save the vector field to a ASCII tabular file.

```
tools.save('exp1_001.txt' , x, y, u, v, invalid_mask)
```

The vector field can be plotted with `display_vector_field`. Vectors with signal-to-noise ratio below the threshold are displayed in red.

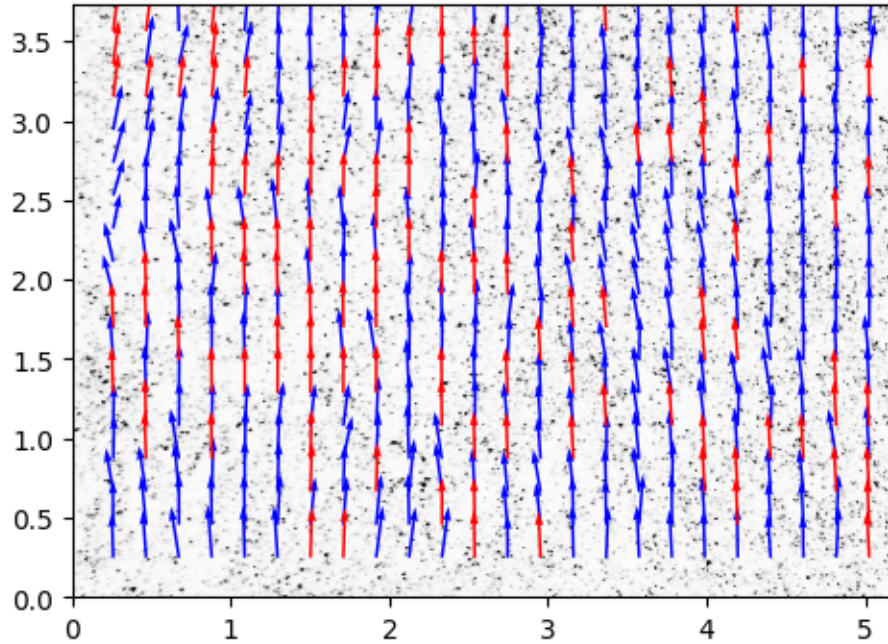
```
import pathlib

fig, ax = plt.subplots(figsize=(9,4))
tools.display_vector_field(
```

```

pathlib.Path('exp1_001.txt'),
ax=ax, scaling_factor=96.52,
scale=50, # scale defines here the arrow length
width=0.0035, # width is the thickness of the arrow
on_img=True, # overlay on the image
image_name= str('./images/exp1_001_b.bmp'),
)
plt.show()

```



Another example

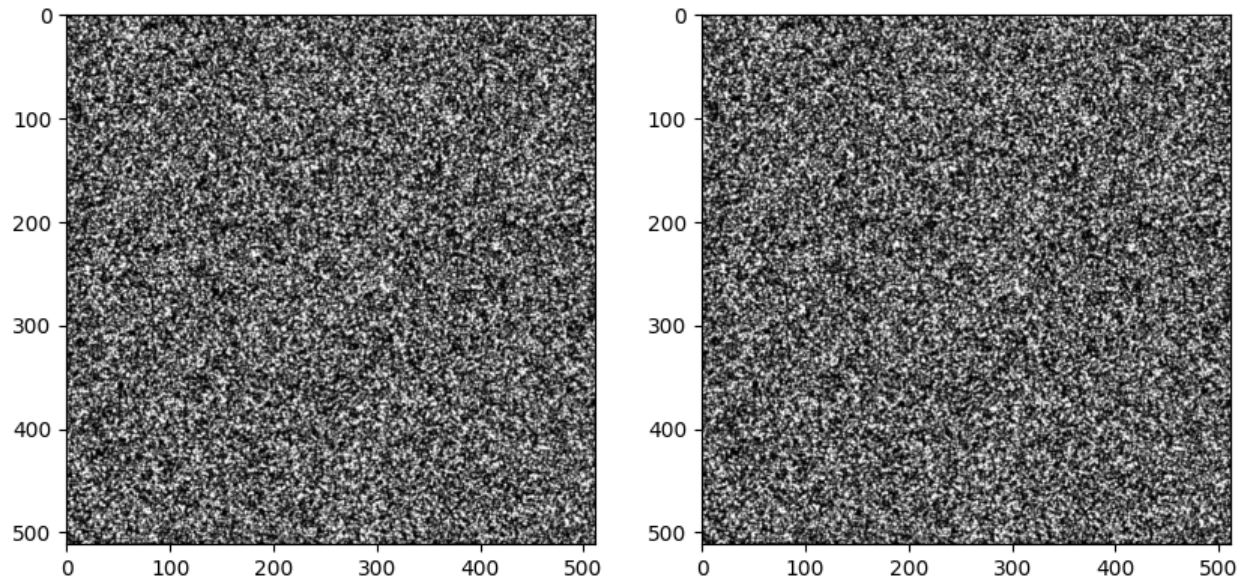
For instance we can access images from the PIV Challenge <http://www.pivchallenge.org/>

```

frame_a = tools.imread('http://www.pivchallenge.org/pub/B/B001_1.tif')
frame_b = tools.imread('http://www.pivchallenge.org/pub/B/B001_2.tif')

fig,ax = plt.subplots(1,2,figsize=(10,8))
ax[0].imshow(frame_a,cmap=plt.cm.gray)
ax[1].imshow(frame_b,cmap=plt.cm.gray)
plt.show()

```



```

winsize = 32 # pixels
searchsize = 64 # pixels, search in image B
overlap = 16 # pixels
dt = 1.0 # sec
u, v, sig2noise = pyprocess.extended_search_area_piv( frame_a.astype(np.int32), frame_b.astype(np.int32))

x, y = pyprocess.get_coordinates(
    image_size=frame_a.shape,
    search_area_size=searchsize,
    overlap=overlap,
)

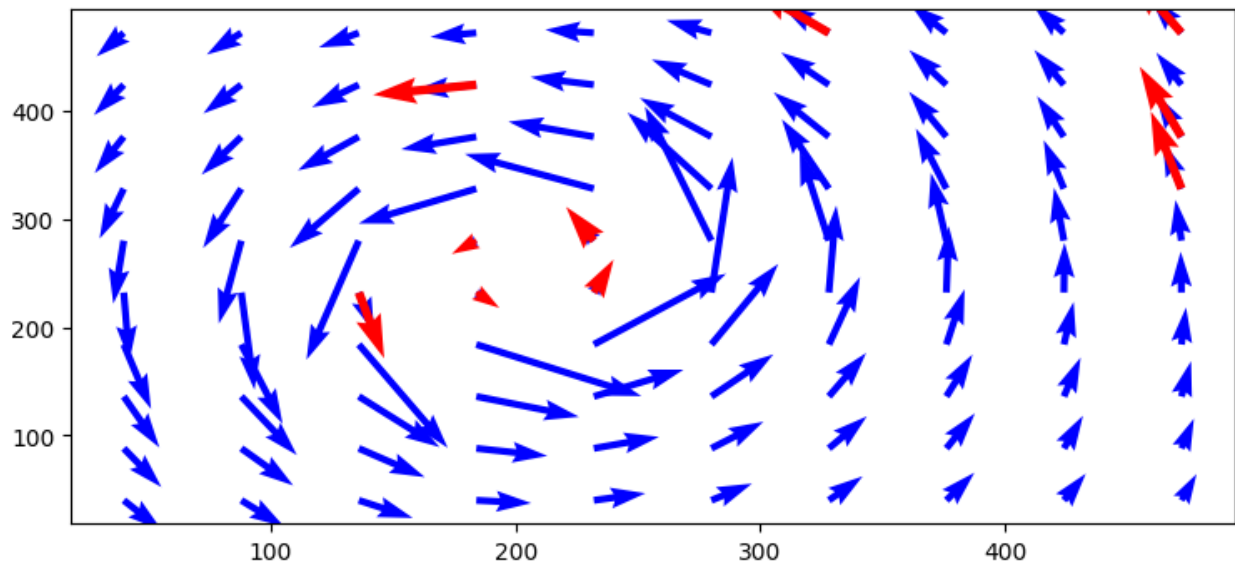
#Post-processing
invalid_mask = validation.sig2noise_val(
    sig2noise,
    threshold = 1.1,
)

u, v = filters.replace_outliers(
    u, v,
    invalid_mask,
    method='localmean',
    max_iter=10,
    kernel_size=2,
)

plt.figure(figsize=(9,4))
plt.quiver(x,y,u,v,color='b')

```

```
plt.quiver(x[invalid_mask],y[invalid_mask],u[invalid_mask],v[invalid_mask],color='r')  
plt.show()
```



Advanced PIV techniques

This page provides an in-depth overview of advanced Particle Image Velocimetry (PIV) techniques implemented in the OpenPIV Python package. It covers window deformation methods, multi-pass processing, vector validation, and image preprocessing techniques that go beyond basic PIV analysis.

How window deformation works

Window deformation is an advanced technique that improves PIV accuracy by deforming interrogation windows based on previous displacement estimates. This is particularly useful for flows with high shear or rotation.

Window deformation iteratively deforms the interrogation windows to account for flow gradients within each window. This helps overcome the limitations of standard PIV which assumes uniform displacement within each interrogation region.

OpenPIV supports two window deformation methods. 1. Symmetric Deformation: Both images are deformed by half the displacement in opposite directions. This is more accurate but computationally intensive. 2. Second Image Deformation: Only the second image is deformed. This is faster but potentially less accurate.

How multi-pass processing works

Multi-pass processing iteratively refines PIV analysis by using results from previous passes to guide subsequent analysis with smaller interrogation windows. This approach significantly improves spatial resolution and accuracy. 1. First pass uses large interrogation windows for robustness 2. Subsequent passes use smaller windows for better spatial resolution 3. Each pass uses results from the previous pass to deform windows

Vector Validation Techniques

Vector validation is crucial for identifying and removing spurious vectors from PIV results. OpenPIV implements several validation methods that can be used alone or in combination:

1. Global Validation (`global_val`): Rejects vectors outside specified displacement ranges.

2. Global Statistical Validation (`global_std`): Rejects vectors that deviate from the mean by more than a specified number of standard deviations.
3. Signal-to-noise Validation (`sig2noise_val`): Rejects vectors with a low signal-to-noise ratio.
4. Local Median Validation (`local_median_val`): Rejects vectors that deviate significantly from their local neighborhood median.
5. Normalized Local Median Validation (`local_norm_median_val`): A normalized version of local median validation that accounts for local flow variations.

Vector Replacement

After validation, identified outliers can be replaced using different methods:

1. `localmean`: Replaces outliers with the local neighborhood mean
2. `disk`: Uses a disk-shaped kernel for replacement
3. `distance`: Weighted average based on distance

Configuring Advanced PIV Analysis

Advanced PIV techniques are configured using the `PIVSettings` class, which provides a centralized way to control all aspects of the analysis.

Key Settings

- `window_sizes`: Tuple of interrogation window sizes for each pass (e.g., (64, 32, 16))
- `overlap`: Tuple of overlap values for each pass (e.g., (32, 16, 8))
- `num_iterations`: Number of PIV passes
- `correlation_method`: Method for correlation (`circular` or `linear`)
- `deformation_method`: Method for window deformation (`symmetric` or `second image`)
- `validation_first_pass`: Whether to validate the first pass
- `replace_vectors`: Whether to replace outliers
- Validation thresholds: `min_max_u_disp`, `min_max_v_disp`, `std_threshold`, `median_threshold`, etc.

Example Usage

Using advanced PIV techniques in OpenPIV involves setting up `PIVSettings` and calling the `windef.piv()` function:

```

# Import necessary modules
from openpiv import windef
import pathlib

image_path = pathlib.Path(r'./images/')

file_list = []
for path in sorted(image_path.rglob('*.tif')):
    print(f'{path.name}')
    file_list.append(path.name)

# Create settings object
settings = windef.PIVSettings()

# Data related settings
settings.filepath_images = image_path # './images/' # Folder with the images to process
settings.save_path = './results/' # Folder for the outputs
# Root name of the output Folder (if any) for Result Files
settings.save_folder_suffix = 'test'
# Format and Image Sequence (see below for more options)
#settings.frame_pattern_a = 'exp1_001_b.bmp'
#settings.frame_pattern_b = 'exp1_001_c.bmp'
# or if you have a sequence:
settings.frame_pattern_a = '*.tif'
# settings.frame_pattern_b = '(1+2),(2+3)'
# settings.frame_pattern_b = '(1+3),(2+4)'
settings.frame_pattern_b = '(1+2),(3+4)'

# Format and Image Sequence
#settings.frame_pattern_a = '*.bmp' # file_list[0]

# settings.frame_pattern_b = file_list[1]
#settings.frame_pattern_b = None

# If you want only one pair
#settings.frame_pattern_a = file_list[0]
#settings.frame_pattern_b = file_list[1]

# Region of interest: (xmin,xmax,ymin,ymax) or 'full' for full image
settings.roi = 'full'

# Configure settings for advanced analysis
settings.window_sizes = (64, 32, 16) # it should be a power of 2
settings.overlap = (32, 16, 8) # This is 50% overlap. In general window size/2 is a good choice

```

```

settings.num_iterations = 3 # select the number of PIV passes
settings.correlation_method = 'circular' # 'circular' or 'linear'
settings.normalized_correlation = False
settings.subpixel_method = 'gaussian' # 'gaussian', 'centroid', 'parabolic'
settings.deformation_method = 'symmetric'
settings.interpolation_order = 3 # order of the image interpolation for the window deformation

# Signal to noise ratio options (only for the last pass)
# It is possible to decide if the S/N should be computed (for the last pass) or not
# If extract_sig2noise==False the values in the signal to noise ratio
# output column are set to NaN
settings.extract_sig2noise = True # 'True' or 'False' (only for the last pass)
settings.sig2noise_method = 'peak2peak' # 'peak2peak' or 'peak2mean'
# select the width of the masked to masked out pixels next to the main peak
settings.sig2noise_mask = 2

# Set vector validation parameters
settings.validation_first_pass = True # choose if you want to do validation of the first pass
# The validation is done at each iteration based on three filters.
# The first filter is based on the min/max ranges. Observe that these values are defined in
# terms of minimum and maximum displacement in pixel/frames.
settings.min_max_u_disp = (-30, 30)
settings.min_max_v_disp = (-30, 30)
# The second filter is based on the global STD threshold
settings.std_threshold = 7 # threshold of the std validation
# The third filter is the median test (not normalized at the moment)
settings.median_threshold = 3 # threshold of the median validation
# Validation based on the signal to noise ratio'
# Note: only available when extract_sig2noise==True and only for the last
# pass of the interrogation
# Options: True or False
settings.sig2noise_threshold = 1.2 # minnum signal to noise ratio that is need for a valid vect

# Outlier replacement or Smoothing options
# Replacment options for vectors which are masked as invalid by the validation
settings.replace_vectors = True # True or False
settings.smoothn = True #Enables smoothing of the displacement field
settings.smoothn_p = 0.5 # This is a smoothing parameter
# select a method to replace the outliers: 'localmean', 'disk', 'distance'
settings.filter_method = 'localmean'
# maximum iterations performed to replace the outliers
settings.max_filter_iteration = 4
settings.filter_kernel_size = 2 # kernel size for the localmean method

settings.scaling_factor = 1 # scaling factor pixel/meter

```

```

settings.dt = 1 # time between to frames (in seconds)

# Output options
# Select if you want to save the plotted vector field: True or False
settings.save_plot = False
# Choose wether you want to see the vectorfield or not :True or False
settings.show_plot = True
settings.scale_plot = 200 # select a value to scale the quiver plot of the vectorfield

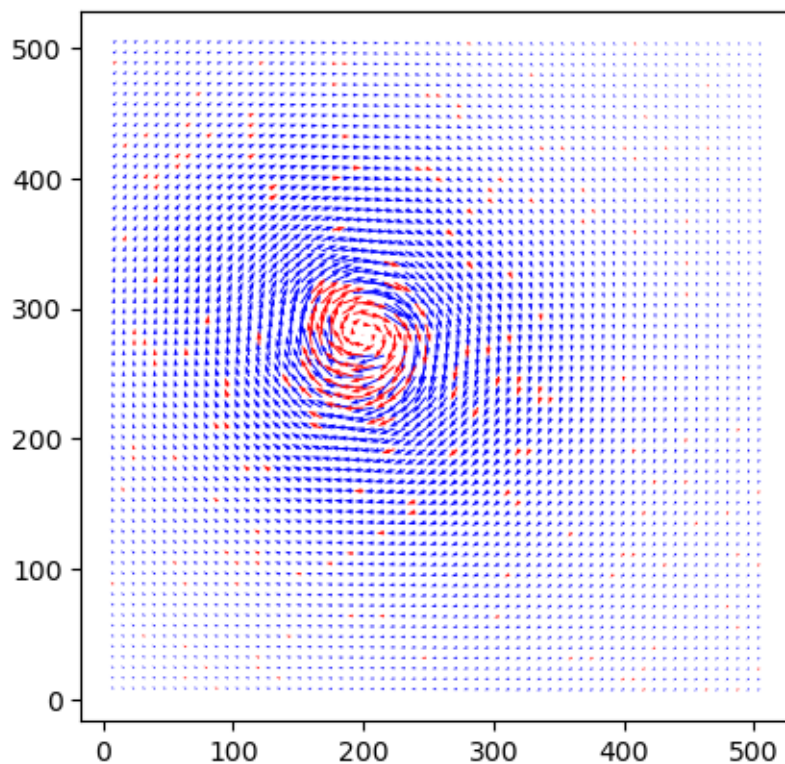
# Run PIV analysis with the given settings
windef.piv(settings)

```

```

B005_1.tif
B005_2.tif
B005_3.tif
B005_4.tif
Saving to results/OpenPIV_results_16_test/field_A0000.txt

```



```

Image Pair 1
B005_1 B005_2
Saving to results/OpenPIV_results_16_test/field_A0001.txt

```

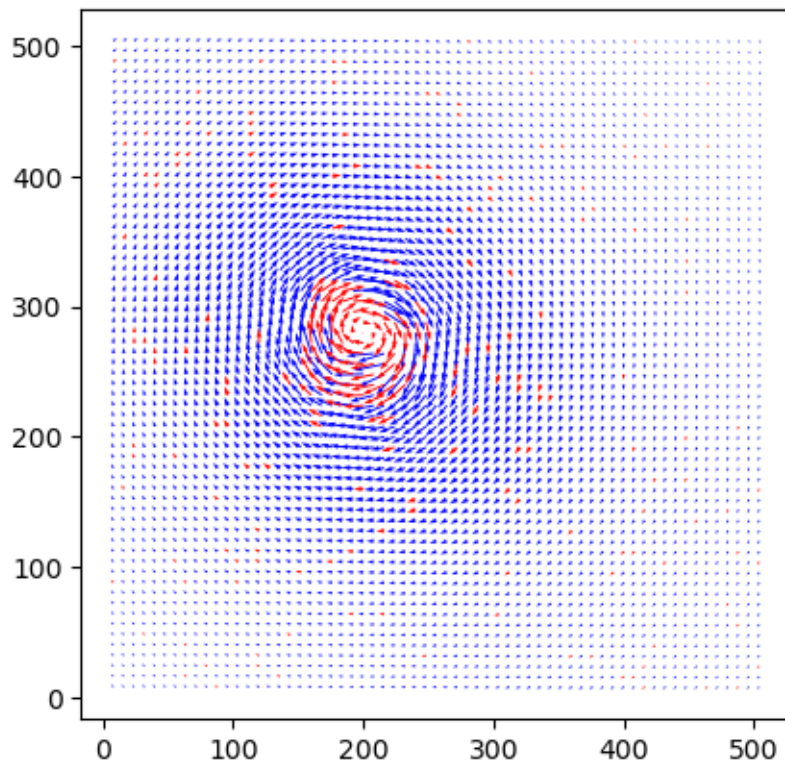


Image Pair 2
B005_3 B005_4

You can afterwards easily access the computed velocity fields as,

```
# Read x, y, u, v variables from the result file
data = np.loadtxt('results/OpenPIV_results_16_test/field_A0000.txt', skiprows=1)
x = data[:, 0]
y = data[:, 1]
u = data[:, 2]
v = data[:, 3]
```

and estimate/show 2d visualizations:

```
# Create grid for 2D visualization
xi = np.unique(x)
yi = np.unique(y)
X, Y = np.meshgrid(xi, yi)
U = u.reshape(len(yi), len(xi))
V = v.reshape(len(yi), len(xi))
L2 = np.sqrt(U**2 + V**2)
```

```

fig_l2, ax_l2 = plt.subplots(figsize=(6,5))
mesh_l2 = ax_l2.pcolormesh(X, Y, L2, shading='auto')
plt.colorbar(mesh_l2, ax=ax_l2, label='L2 norm')
ax_l2.set_title('L2 norm of velocity')
ax_l2.set_xlabel('x')
ax_l2.set_ylabel('y')
fig_l2.tight_layout()

plt.show()

```

Summary

Advanced PIV techniques in OpenPIV provide powerful tools for improving accuracy and spatial resolution in PIV analysis. Window deformation and multi-pass processing address limitations of basic PIV analysis, while comprehensive validation methods ensure reliable vector fields. These techniques are particularly valuable for complex flows with high shear, rotation, or spatial gradients.

Assignment. Now that you are familiar with the advanced PIV techniques, you will be asked to analyze some datasets of a particular flow: a Lamb-Oseen vortex flow which is often used for validation due to its well-known analytical solutions. You will generate series of images of this vortex flow with different particle densities and/or different particles diameters and analyze them using the advanced PIV techniques covered in this tutorial. A [package](#) of Python scripts is provided to help you with the generation.

Give an evaluation of the velocity vector fields for every generated test case. Give some insights about the PIV parameters used for the analyses. Evaluate the precision of your estimates with regards to the exact location of the particles as given in `parameters.h5`.

Practice

Measurements

You will conduct experiments with the aim of determining the velocity field of a flow using PIV. The experiments are carried out in a recirculating-water glass tank measuring $30 \times 35 \times 200 \text{ cm}^3$. Its modest volume is well-suited for the space constraints of a practical laboratory exercise. However, you will notice that it has **consequences on the established flow** that you will analyze. The pump can produce an adjustable flow rate of up to approximately $200 \text{ m}^3/\text{h}$, thus creating a controlled recirculating flow in the aquarium.

Experimental setup

Seeding particles

The seeding particles used are polyamide particles (PSP-50, ref. 9080A5011). These particles are produced by polymerization. They are round, but not perfectly spherical. The size distribution (diameter) of each particle is between 30 and 70 μm , with an average of 50 μm . Their density is 1.03 g/cm^3 , very close to that of water, which therefore limits their sedimentation. Their refractive index is 1.5.

Light source

Illumination is provided by the Fiber-Lite® Mi-LED light generator (Dolan-Jenner). This system delivers white light with a color temperature of 5000 K and represents a modern and economical alternative to conventional 150 W halogen sources.

Image recording

The images are acquired by the FlowSense 2M-165 camera, which connects directly to the computer via a USB 3 port, thus integrating the system directly within the computer without requiring an additional acquisition card. This camera offers a resolution of 1920×1200 pixels (2.3 megapixels), with a maximum acquisition frequency of 160 frames per second. It is therefore well-suited for time-resolved PIV for slow to moderate flows. Its pixel size is 5.9 μm , and the quantum efficiency of its sensor is greater than 70%, particularly adapted to the wavelengths of green light from lasers or LEDs. It uses a CMOS (Complementary Metal-Oxide-Semiconductor) sensor, which is widely used in modern PIV systems for its high acquisition rate.

Acquisition protocol

This is a **quick-reference checklist** meant to be read once before your rotation slot, not a tutorial. The station is shared: one binome at a time. **Bring a USB key.**

Before your slot: target realizations per series

- The mean velocity field estimated from N independent realizations converges as roughly $\sigma/\sqrt{N}$: doubling N only buys you a $\sqrt{2}$ reduction in noise. Default: **~150 independent pairs** per flow/regime for the batch statistics — going from 150 to 500 only buys a factor ~ 1.8 in precision for over 3x the capture/transfer time.

One-time setup

- **Calibration image.** With the empty aquarium (no obstacle) and the illumination sheet in place, image the dot target held flat in the sheet plane. Capture **one single frame**. Note which two graduations you can read clearly and their spacing in mm.

Per case (flow $\times$ regime)

1. Mount the obstacle (cylinder / step insert) or remove it (free stream).
2. Set the pump regime (low/mid/high). Wait for the flow to visibly settle before capturing.
3. **Choose Δt** (interframe time): a particle should move roughly a **quarter of the final interrogation window** between the two frames. Too large a Δt loses particles out-of-plane; too small gives displacements too close to the pixel-locking regime discussed in the Lecture. **Write down the Δt used** (1 / frame rate, in seconds), there is no way to recover it from the images themselves afterwards.
4. Capture your target number of images.
5. Copy the raw TIFF frames for this case onto a folder on your USB key (one folder per flow/regime — you'll sort and convert them, back at your laptop — see the Appendix at the end of this notebook for the conversion).

Summary

During this practical session, you will get familiarized with a simple experimental setup designed to record particle images of a flow for which the velocity field is to be estimated.

Assignment. You will be asked to study three particular flows:

1. a free flow (without any obstacle)
2. the flow that develops around/behind a cylinder
3. the flow that develops behind a step

each of which with three flow regimes, i.e. for three different flow rates as given by the pump. Record sufficiently substantial samples to allow you to subsequently conduct statistical studies.

An in-depth analysis of the obtained images will be expected. Among the points to be discussed, without being exhaustive, should include: the average size of the particle images, their density, and the dynamic range of the gray levels. To do this, you could for example use the **regionprops** library adapted from Matlab:

```
# Import necessary modules
import numpy as np
import matplotlib.pyplot as plt
#matplotlib inline
from openpiv import tools
from skimage.measure import label, regionprops

# Load a sample image
image = tools.imread('your image.tif')
fig, ax = plt.subplots(figsize=(12, 10))
ax.imshow(image, cmap=plt.cm.gray)

# Use a threshold to segment bright particles
threshold = np.percentile(image, 90) # adjust percentile as needed
binary_img = image > threshold

label_img = label(binary_img, background=0) # adjust background as needed
regions = regionprops(label_img, intensity_image=image)

for props in regions:
    # Use weighted centroid for better accuracy
    y0, x0 = props.weighted_centroid
    ax.plot(x0, y0, marker='x', color='r', markersize=6)

plt.show()
```

You will then work on the collected databases by implementing a method for evaluating velocity fields using OpenPIV. You will detail the method used, emphasizing the choice of parameters for the PIV analysis.

Finally, you will develop an analysis of the obtained velocity fields based on, among other things, considerations discussed in class with Mr. Druault.

Appendix: converting your images

Use the function below to convert the raw TIFF frames of a folder into PNG. Non-8-bit TIFFs are rescaled to 8-bit first — this is also what makes the PNGs much smaller than the raw TIFFs.

Call `convert_to_png(src_dir, dst_dir)` once per folder you captured. Pass `prefix="..."` if you also want to rename the files: the trailing number of each file name is kept and everything before it is replaced by your prefix.

```
import pathlib
import re
import numpy as np
import imageio.v3 as iio

def convert_to_png(src_dir, dst_dir=None, prefix=None):
    """Convert every .tif/.tiff image of `src_dir` to a .png in `dst_dir`
    (default: next to the originals).

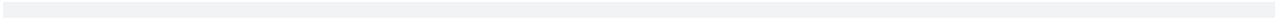
    prefix: if given, rename the files -- the trailing number of each name
    is kept (same zero-padding), everything before it is replaced by `prefix`.
    """
    src_dir = pathlib.Path(src_dir)
    dst_dir = pathlib.Path(dst_dir) if dst_dir else src_dir
    files = sorted(f for f in src_dir.iterdir() if f.suffix.lower() in ('.tif', '.tiff'))
    if not files:
        print(f"No .tif/.tiff files found in {src_dir}")
        return
    dst_dir.mkdir(parents=True, exist_ok=True)

    for f in files:
        img = iio.imread(f)
        if img.dtype != np.uint8:          # rescale to 8 bit -> much smaller PNG
            img = img.astype(np.float64)
            img -= img.min()
            if img.max() > 0:
                img = img / img.max() * 255.0
            img = img.astype(np.uint8)

        stem = f.stem
        if prefix is not None:
            m = re.match(r'^(.*?)(\d+)$', stem)
            stem = f"{prefix}-{int(m.group(2)):0{len(m.group(2))}d}" if m else f"{prefix}-{stem}"

        iio.imwrite(dst_dir / f"{stem}.png", img)
        print(f"{f.name} -> {stem}.png")
    print(f"Done: {len(files)} image(s) converted in {dst_dir}")

# Example -- adapt the paths to your USB key:
# convert_to_png("/Volumes/USB_KEY/cylinder_mid", "data/cylinder/mid")
# convert_to_png("/Volumes/USB_KEY/calib", "data/calibration")
```



Processing

This notebook turns the “Measurements” assignment (Chapter 4) into a runnable, guided pipeline, applied to **your own images**.

It is written as a **generic template**: the same code processes all three flows studied during the lab sessions — a free stream, the wake of a cylinder, and the flow behind a backward-facing step — at the three pump regimes. This is not meant to be a complete overview.

About the examples shown below. This statement was written before any student data existed, so the walkthrough is illustrated with a synthetic placeholder dataset (`synthetic_piv.py`) — just to orient you towards the analyses expected at each step. Set `DATA_SOURCE` to “real” in §1 and everything below runs the same way on your own images instead; the few places marked `are` are where real data needs a value you measure yourself rather than one we generated.

This builds directly on:

- Chapter 1-2 (cross-correlation, `extended_search_area_piv`)
- Chapter 3 (multi-pass, window deformation, `PIVSettings` / `windef.piv`, vector validation)

```
# Standard imports
import pathlib, shutil, time, io, contextlib
import numpy as np
import matplotlib.pyplot as plt
import imageio.v3 as iio

from openpiv import tools, windef
from skimage.measure import label, regionprops

@contextlib.contextmanager
def quiet():
    # OpenPIV's windef.piv() is very verbose!
    with contextlib.redirect_stdout(io.StringIO()):
        yield

# synthetic data generator for the examples in this notebook
import sys
sys.path.insert(0, '.')
from synthetic_piv import (
    render_pair, render_sequence, velocity_field, solid_mask,
```

```

    reynolds_number, shedding_frequency,
    FLOWS, REGIMES, IMG_SHAPE, CAL, FOV_MM,
    CYL_X_MM, CYL_Y_MM, CYL_D_MM, STEP_X_MM, STEP_H_MM, STEP_LR_MM,
)

np.random.seed(0)
%matplotlib inline

```

Dataset

FLOWS are the three configurations studied in the lab ("free", "cylinder", "step"), each run at three regimes ("low", "mid", "high") corresponding to the three pump settings.

Each sequence of images must sit in `data/real/<flow>/<regime>/` as a series of `.png` files.

```

# =====
# DATA SOURCE -- this is the one switch to flip once your own images are
# ready.
# =====
DATA_SOURCE = "synthetic" # "synthetic" or "real"

# real only: time between the two consecutive frames.
DT_PIV_REAL = 0.001
DT_FRAME_REAL = {"low": None, "mid": None, "high": None}

N_REALIZATIONS_SYNTHETIC = 10 # only used to generate the synthetic placeholder
DT_PIV_SYNTHETIC = 0.06 # s, only used to generate the synthetic placeholder

# One folder per case: DATA_ROOT/<flow>/<regime>/*.png.
DATA_ROOT = pathlib.Path('data/synthetic' if DATA_SOURCE == "synthetic" else 'data/real')
DT_PIV = DT_PIV_SYNTHETIC if DATA_SOURCE == "synthetic" else DT_PIV_REAL

def frames_of(flow, regime):
    """Image sequence of one case, in natural order."""
    return tools.natural_sort(list((DATA_ROOT / flow / regime).glob('*.png'))))

def generate_synthetic_dataset():
    """Write the placeholder sequences: for each case, 2 x N_REALIZATIONS_SYNTHETIC
    frames (A, B, A, B, ...) as DATA_ROOT/<flow>/<regime>/frame_NNNN.png."""
    if DATA_ROOT.exists():
        shutil.rmtree(DATA_ROOT)
    for flow in FLOWS:
        for regime in REGIMES:
            out_dir = DATA_ROOT / flow / regime

```

```

        out_dir.mkdir(parents=True)
        rng_seed_base = hash((flow, regime)) % 10_000
        for i in range(N_REALIZATIONS_SYNTHETIC):
            t_sample = np.random.default_rng(rng_seed_base + i).uniform(0, 10)
            a, b, _ = render_pair(flow, regime, t=t_sample, dt=DT_PIV_SYNTHETIC,
                                seed=rng_seed_base + i)
            iio.imwrite(str(out_dir / f'frame_{2 * i:04d}.png'), a)
            iio.imwrite(str(out_dir / f'frame_{2 * i + 1:04d}.png'), b)

np.random.seed(0)
if DATA_SOURCE == "synthetic":
    generate_synthetic_dataset()

for flow in FLOWS:
    for regime in REGIMES:
        n_frames = len(frames_of(flow, regime))
        if n_frames == 0:
            raise FileNotFoundError(
                f"No .png frames found in {DATA_ROOT / flow / regime}. Convert your "
                f"recorded sequence for this case there first (see the Appendix of Chapter 4)."
            )
        if n_frames % 2:
            raise ValueError(
                f"{DATA_ROOT / flow / regime}: odd number of frames ({n_frames}) -- "
                f"pairing with '(1+2),(3+4)' ($5) needs an even number, remove the extra one."
            )
        print(f" {flow:10s} {regime:6s} -> {n_frames} frames")

IMAGE_SHAPE = iio.imread(frames_of(FLOWS[0], list(REGIMES)[0])[0]).shape
print(f"image shape: {IMAGE_SHAPE}")

```

image shape: (192, 384)

```

print(f"{'flow':10s} {'regime':6s} {'U0 (mm/s)':>10s} {'Re':>8s} {'f_shed (Hz)':>12s}")
for flow in FLOWS:
    for regime in REGIMES:
        U0 = 60.0 * REGIMES[regime]
        f_str = f"{shedding_frequency(regime):.3f}" if flow == 'cylinder' else "-"
        print(f"{flow:10s} {regime:6s} {U0:10.1f} {reynolds_number(flow, regime):8.0f} {f_str:>12s}")

```

flow	regime	U0 (mm/s)	Re	f_shed (Hz)
free	low	36.0	540	-
free	mid	60.0	900	-

free	high	90.0	1350	-
cylinder	low	36.0	432	0.600
cylinder	mid	60.0	720	1.000
cylinder	high	90.0	1080	1.500
step	low	36.0	540	-
step	mid	60.0	900	-
step	high	90.0	1350	-

```
# Preview one image pair per flow (mid regime), straight from disk
fig, axs = plt.subplots(1, 3, figsize=(13, 4))
for ax, flow in zip(axs, FLOWS):
    a = iio.imread(frames_of(flow, 'mid')[0])
    ax.imshow(a, cmap='gray', vmin=0, vmax=255)
    ax.set_title(f'{flow} (frame A, mid regime)')
    ax.set_xlabel('x (px)'); ax.set_ylabel('y (px)')
plt.tight_layout()
plt.show()
```

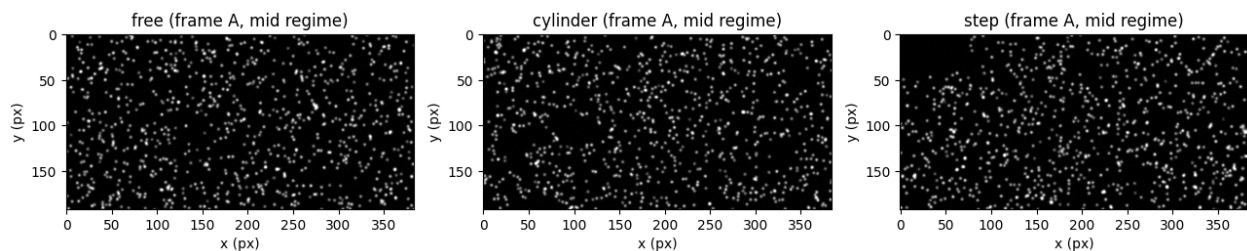


Image-quality diagnostics

Following Chapter 4, before running any correlation we look at the raw images: particle image size, density and grey-level dynamic range. The code below is the same `regionprops`-based approach, applied here to one of our synthetic frames — apply it unchanged to a sample of your own images.

```
sample = iio.imread(frames_of('cylinder', 'mid')[0])

threshold = np.percentile(sample, 90)
binary_img = sample > threshold
label_img = label(binary_img, background=0)
regions = regionprops(label_img, intensity_image=sample)

diam_px = np.array([2 * np.sqrt(r.area / np.pi) for r in regions])
n_particles = len(regions)
density = n_particles / (sample.shape[0] * sample.shape[1]) # particles / px^2
```

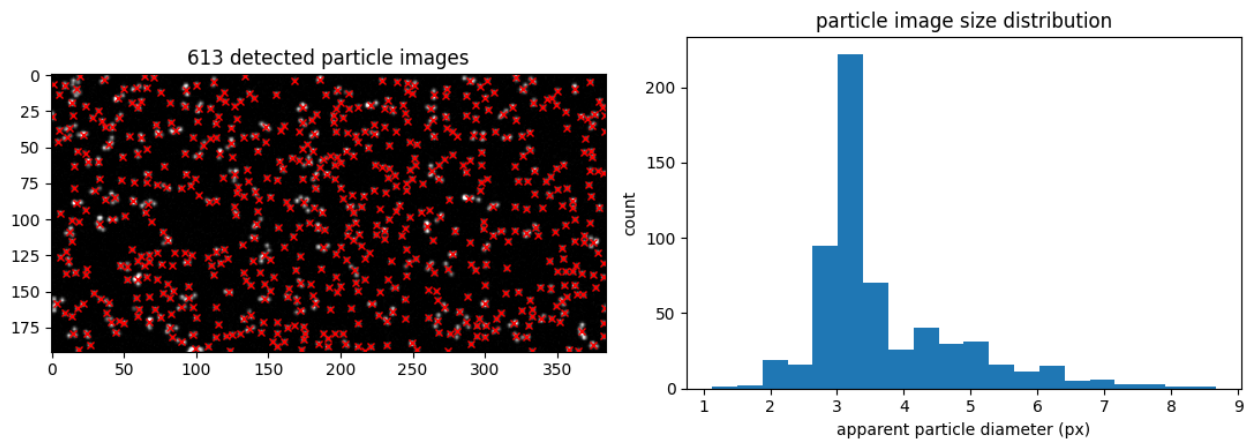
```

fig, axs = plt.subplots(1, 2, figsize=(11, 4))
axs[0].imshow(sample, cmap='gray')
for r in regions:
    y0, x0 = r.weighted_centroid
    axs[0].plot(x0, y0, marker='x', color='r', markersize=5)
axs[0].set_title(f'{n_particles} detected particle images')

axs[1].hist(diam_px, bins=20)
axs[1].set_xlabel('apparent particle diameter (px)')
axs[1].set_ylabel('count')
axs[1].set_title('particle image size distribution')
plt.tight_layout()
plt.show()

print(f"seeding density      : {density*1e3:.3f} 10-3 particles px2")
print(f"mean apparent diameter : {diam_px.mean():.2f} px")
print(f"dynamic range           : {sample.min()} - {sample.max()} (8-bit)")

```



```

seeding density      : 8.314 10-3 particles px2
mean apparent diameter : 3.76 px
dynamic range         : 0 - 255 (8-bit)

```

On real images, report these numbers for each of your recordings and compare with what you obtained in the theory part.

Masking

Within an obstacle part of the field of view (cylinder or step): no particles, no valid vectors there. OpenPIV's `windef.PIVSettings.static_mask` accepts a boolean array (`True` = masked) the same size as the images.

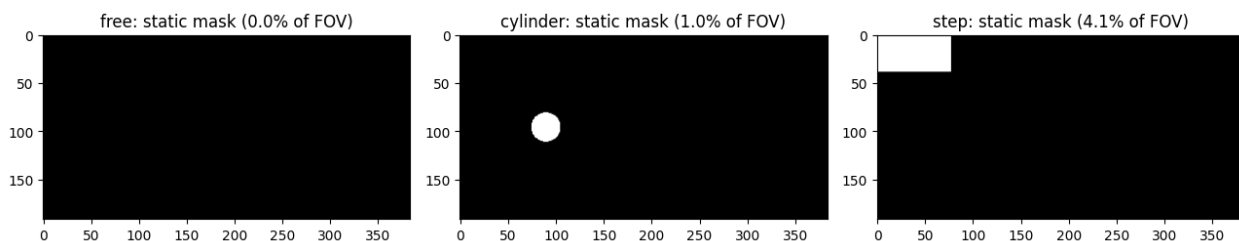
On real images, `mask_for(flow)` below reads off the obstacle geometry (here, in mm but can be written in pxl) from a representative frame. `openpiv.preprocess.dynamic_masking` is an alternative if a simple circle/rectangle isn't a good enough fit (worth trying if the parametric mask leaves visible artifacts at the obstacle edge).

```
def mask_for(flow):
    ny, nx = IMAGE_SHAPE
    xs_px, ys_px = np.meshgrid(np.arange(nx), np.arange(ny))
    x_mm, y_mm = xs_px / scaling_factor, ys_px / scaling_factor

    if DATA_SOURCE == "synthetic":
        return solid_mask(flow, x_mm, y_mm)

    # Real, parametric mask -- fill in from a representative frame ($1/$2):
    if flow == 'cylinder':
        cx_mm, cy_mm, d_mm = 35.0, y_mm.max() / 2, 12.0 # centre (x,y), diameter, mm
        return (x_mm - cx_mm) ** 2 + (y_mm - cy_mm) ** 2 < (d_mm / 2) ** 2
    if flow == 'step':
        step_x_mm, step_h_mm = 30.0, 15.0 # step location, height, mm
        return (x_mm < step_x_mm) & (y_mm < step_h_mm)
    return np.zeros((ny, nx), dtype=bool) # free stream: nothing to mask

fig, axs = plt.subplots(1, 3, figsize=(13, 3.2))
for ax, flow in zip(axs, FLOWS):
    m = mask_for(flow)
    ax.imshow(m, cmap='gray')
    ax.set_title(f'{flow}: static mask ({m.mean()*100:.1f}% of FOV)')
plt.tight_layout()
plt.show()
```



Batch multi-pass processing

We now run the advanced pipeline from Chapter 3 (`PIVSettings` + `windef.piv`) over a **batch of independent realizations** for every flow/regime combination.

Pairs are built from the sequence with `settings.frame_pattern_b = '(1+2),(3+4)'` (frames 1+2, then 3+4, ...), as in Chapter 3.

```
# Default window (64->32->16 px), to be adapted.
DEFAULT_WINDOWSIZE, DEFAULT_OVERLAP = (64, 32, 16), (32, 16, 8)
WINDOWSIZE = {"step": (48, 24, 12)}
OVERLAP = {"step": (24, 12, 6)}

def windows_for(flow):
    return WINDOWSIZE.get(flow, DEFAULT_WINDOWSIZE), OVERLAP.get(flow, DEFAULT_OVERLAP)

RESULTS_ROOT = pathlib.Path('results')
if RESULTS_ROOT.exists():
    shutil.rmtree(RESULTS_ROOT)

def make_piv_settings(image_dir, save_dir, mask, dt, suffix, windowsize, overlap,
                    pattern_a='*.png', pattern_b='(1+2),(3+4)'):
    s = windef.PIVSettings()
    s.filepath_images = image_dir
    s.save_path = save_dir
    s.save_folder_suffix = suffix
    s.frame_pattern_a = pattern_a
    s.frame_pattern_b = pattern_b
    s.roi = 'full'
    s.static_mask = mask
    s.windowsize = windowsize
    s.overlap = overlap
    s.num_iterations = len(windowsize)
    s.correlation_method = 'circular'
    s.subpixel_method = 'gaussian'
    s.deformation_method = 'symmetric'
    s.validation_first_pass = True
    s.min_max_u_disp = (-200, 200)
    s.min_max_v_disp = (-200, 200)
    s.std_threshold = 6
    s.median_threshold = 3
    s.sig2noise_threshold = 1.1
    s.replace_vectors = True
    s.smoothn = True
    s.smoothn_p = 0.5
    s.filter_method = 'localmean'
    s.max_filter_iteration = 4
    s.filter_kernel_size = 2
    s.scaling_factor = scaling_factor # px -> mm
    s.dt = dt
    s.show_plot = False
```

```

s.save_plot = False
return s

t0 = time.time()
for flow in FLOWS:
    mask = mask_for(flow)
    windowsizes, overlap = windows_for(flow)
    for regime in REGIMES:
        image_dir = DATA_ROOT / flow / regime
        n_pairs = len(frames_of(flow, regime)) // 2

        settings = make_piv_settings(
            image_dir=image_dir,
            save_dir=RESULTS_ROOT / flow / regime,
            mask=mask, dt=DT_PIV, suffix='batch',
            windowsizes=windowsizes, overlap=overlap,
        )
        with quiet():
            windef.piv(settings)
        print(f" {flow:10s} {regime:6s} -> {n_pairs} pairs processed "
              f"(final window {windowsizes[-1]} px)")

print(f"\nAll flow/regime cases processed in {time.time()-t0:.1f} s")

```

```

free      low    -> 10 pairs processed (final window 16 px)
free      mid    -> 10 pairs processed (final window 16 px)
free      high   -> 10 pairs processed (final window 16 px)
cylinder  low    -> 10 pairs processed (final window 16 px)
cylinder  mid    -> 10 pairs processed (final window 16 px)
cylinder  high   -> 10 pairs processed (final window 16 px)
step      low    -> 10 pairs processed (final window 12 px)
step      mid    -> 10 pairs processed (final window 12 px)
step      high   -> 10 pairs processed (final window 12 px)

```

All flow/regime cases processed in 15.0 s

Quick quality control

Before moving to the analysis notebook, check how many vectors needed validation/replacement in each case — a first, cheap indicator of where the PIV parameters (window sizes, thresholds) might need adjusting.

```

def batch_invalid_fraction(flow, regime):
    windowsizes, _ = windows_for(flow)
    folder = RESULTS_ROOT / flow / regime / f'OpenPIV_results_{windowsizes[-1]}_batch'
    fracs = []
    for f in sorted(folder.glob('field_A*.txt')):
        data = np.loadtxt(f, skiprows=1)
        flags = data[:, 4]
        fracs.append(np.mean(flags != 0))
    return np.mean(fracs)

print(f"{'flow':10s} {'regime':6s} {'mean invalid-vector fraction':>28s}")
for flow in FLOWS:
    for regime in REGIMES:
        frac = batch_invalid_fraction(flow, regime)
        print(f"{'flow':10s} {'regime':6s} {'frac*100:27.1f} %")

```

flow	regime	mean invalid-vector fraction
free	low	0.8 %
free	mid	1.3 %
free	high	1.3 %
cylinder	low	1.1 %
cylinder	mid	1.7 %
cylinder	high	2.6 %
step	low	4.2 %
step	mid	5.6 %
step	high	9.4 %

```

def convergence_curve(flow, regime):
    """Running mean of the field-averaged streamwise velocity vs. number of
    realizations included, in processing order."""
    import warnings
    windowsizes, _ = windows_for(flow)
    folder = RESULTS_ROOT / flow / regime / f'OpenPIV_results_{windowsizes[-1]}_batch'
    files = sorted(folder.glob('field_A*.txt'))
    Us, running_means = [], []
    for f in files:
        data = np.loadtxt(f, skiprows=1)
        u, mask_ = data[:, 2], data[:, 5]
        u = np.where(mask_ > 0, np.nan, u)
        Us.append(u)
        with warnings.catch_warnings():
            warnings.simplefilter('ignore', category=RuntimeWarning)
            running_means.append(np.nanmean(np.stack(Us)))
    return np.arange(1, len(files) + 1), np.array(running_means)

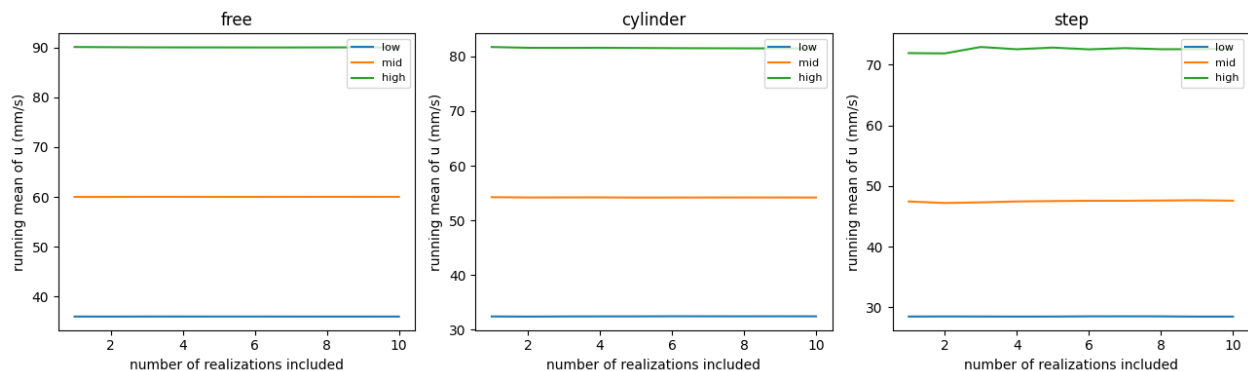
```

```

fig, axs = plt.subplots(1, len(FLOWS), figsize=(13, 4), sharex=False)
for ax, flow in zip(axs, FLOWS):
    for regime in REGIMES:
        n, running_mean = convergence_curve(flow, regime)
        ax.plot(n, running_mean, label=regime)
        ax.set_xlabel('number of realizations included')
        ax.set_ylabel('running mean of u (mm/s)')
        ax.set_title(flow)
        ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

CONVERGENCE_THRESHOLD = 0.02 # flag if the mean is still moving by more than 2%
                             # of its own value over the last 10% of realizations
print(f"{'flow':10s} {'regime':6s} {'N':>4s} {'last-10% relative change':>26s}")
for flow in FLOWS:
    for regime in REGIMES:
        n, running_mean = convergence_curve(flow, regime)
        if len(n) >= 10:
            tail = max(1, len(n) // 10)
            rel_change = abs(running_mean[-1] - running_mean[-1 - tail]) / (abs(running_mean[-1 - tail]) + 1e-10)
            flag = ' <-- consider more realizations in session 2' if rel_change > CONVERGENCE_THRESHOLD else ''
            print(f"{'flow':10s} {'regime':6s} {'len(n)':4d} {'rel_change*100:25.2f}%{flag}")
        else:
            print(f"{'flow':10s} {'regime':6s} {'len(n)':4d} {'(need >= 10 realizations to check)':>26s}")

```



flow	regime	N	last-10% relative change
free	low	10	0.01%
free	mid	10	0.00%
free	high	10	0.00%
cylinder	low	10	0.01%
cylinder	mid	10	0.02%

cylinder	high	10	0.01%
step	low	10	0.00%
step	mid	10	0.16%
step	high	10	0.14%

Summary & next step

We now have, under `results/<flow>/<regime>/`: `OpenPIV_results_<final window>_batch/field_A0000..NNN`
— one file per realization, ready for statistics (mean, fluctuations, turbulence intensity, ...).

If you decide in session 2 that a given flow/regime needs more realizations, re-acquire just that case, replace its frames under `data/real/<flow>/<regime>/`, and re-run from §1 onward.

Analysis

This notebook turns the validated velocity fields under `results/<flow>/<regime>/` into the kind of analysis expected for the assignment — mean/fluctuating fields, vorticity, wake diagnostics, vortex-shedding frequency, and comparisons across regimes and flows.

About the examples shown below. Switch to your own images — this notebook only reads whatever `results/` folder it produced, whatever images it came from.

```
import pathlib
import numpy as np
import matplotlib.pyplot as plt

import sys
sys.path.insert(0, '.')
from synthetic_piv import (
    FLOWS, REGIMES, CYL_X_MM, CYL_Y_MM, CYL_D_MM,
    STEP_X_MM, STEP_H_MM, STEP_LR_MM, ST_TARGET,
    reynolds_number, shedding_frequency,
)

RESULTS_ROOT = pathlib.Path('results')

# Must mirror notebook 5 exactly: $1 DATA_SOURCE, $4 mask geometry,
# $1/$6 DT_FRAME_REAL.
DATA_SOURCE = "synthetic" # "synthetic" or "real"
CYL_GEOM_REAL = dict(cx_mm=35.0, cy_mm=None, d_mm=12.0) # match notebook 5 $4
STEP_GEOM_REAL = dict(x_mm=30.0, h_mm=15.0, lr_mm=None) # match notebook 5 $4 (lr_mm: desi
DT_FRAME_REAL = {"low": None, "mid": None, "high": None} # match notebook 5 $1

if DATA_SOURCE == "synthetic":
    CYL_X, CYL_Y, CYL_D = CYL_X_MM, CYL_Y_MM, CYL_D_MM
    STEP_X, STEP_H, STEP_LR = STEP_X_MM, STEP_H_MM, STEP_LR_MM
else:
    CYL_X, CYL_Y, CYL_D = CYL_GEOM_REAL['cx_mm'], CYL_GEOM_REAL['cy_mm'], CYL_GEOM_REAL['d_mm']
    STEP_X, STEP_H, STEP_LR = STEP_GEOM_REAL['x_mm'], STEP_GEOM_REAL['h_mm'], STEP_GEOM_REAL['lr_mm']

%matplotlib inline
```

1. Loading a batch into 2D fields

Each `field_A####.txt` file has columns `x`, `y`, `u`, `v`, `flags`, `mask` (see Chapter 3). We reshape each realization onto its (regular) grid and stack them so we can compute statistics across realizations at every grid point. Points that fall inside the static mask (`mask == 1`, e.g. inside the cylinder or the step) are set to `NaN` so they do not pollute the statistics.

```
DEFAULT_WINDOWSIZE = (64, 32, 16)
WINDOWSIZE = {"step": (48, 24, 12)}

def batch_folder(flow, regime):
    final_window = WINDOWSIZE.get(flow, DEFAULT_WINDOWSIZE)[-1]
    return RESULTS_ROOT / flow / regime / f'OpenPIV_results_{final_window}_batch'

def load_batch(flow, regime):
    """Returns (X, Y, U, V), or None if this case hasn't been processed yet."""
    folder = batch_folder(flow, regime)
    files = sorted(folder.glob('field_A*.txt'))
    if not files:
        return None
    Us, Vs = [], []
    x = y = None
    for f in files:
        data = np.loadtxt(f, skiprows=1)
        xf, yf, u, v, flags, mask = data.T
        if x is None:
            x, y = xf, yf
        u = np.where(mask > 0, np.nan, u)
        v = np.where(mask > 0, np.nan, v)
        Us.append(u)
        Vs.append(v)
    xi, yi = np.unique(x), np.unique(y)
    shape = (len(yi), len(xi))
    U = np.stack(Us).reshape(-1, *shape) # (n_realizations, ny, nx)
    V = np.stack(Vs).reshape(-1, *shape)
    X, Y = np.meshgrid(xi, yi)
    return X, Y, U, V

def mean_std(U, V):
    import warnings
    with warnings.catch_warnings():
        warnings.simplefilter('ignore', category=RuntimeWarning)
        with np.errstate(invalid='ignore'):
            Umean, Vmean = np.nanmean(U, axis=0), np.nanmean(V, axis=0)
            Ustd, Vstd = np.nanstd(U, axis=0), np.nanstd(V, axis=0)
```

```

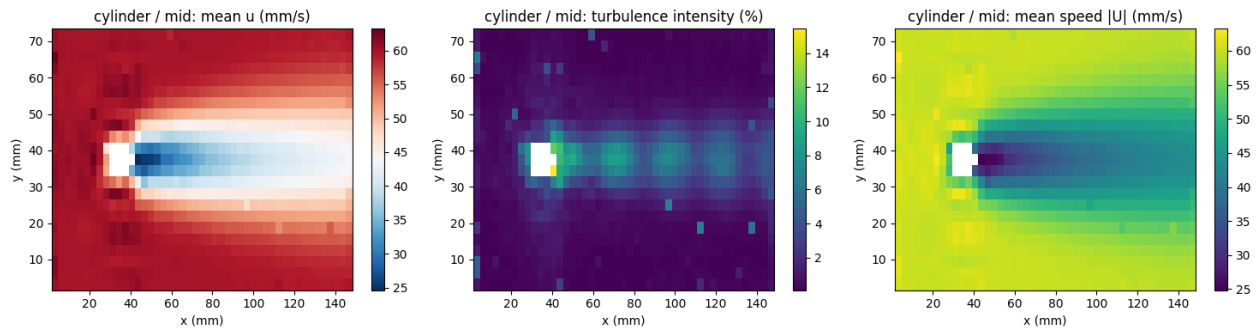
speed_mean = np.sqrt(Umean**2 + Vmean**2)
tke_like = 0.5 * (Ustd**2 + Vstd**2) # ~ turbulent kinetic energy (per unit mass)
return Umean, Vmean, Ustd, Vstd, speed_mean, tke_like

def first_available_case():
    """First (flow, regime) with processed data, used as a demo case if
    ('cylinder', 'mid') itself isn't ready yet."""
    for flow in FLOWS:
        for regime in REGIMES:
            if load_batch(flow, regime) is not None:
                return flow, regime
    return None

demo_case = ('cylinder', 'mid') if load_batch('cylinder', 'mid') is not None else first_available_case()
if demo_case is None:
    print("No processed case found yet -- run notebook 5 first, or wait for mutualized data.")
else:
    demo_flow, demo_regime = demo_case
    if demo_case != ('cylinder', 'mid'):
        print(f"('cylinder', 'mid') not available yet -- using {demo_flow}/{demo_regime} instead")
    X, Y, U, V = load_batch(demo_flow, demo_regime)
    Umean, Vmean, Ustd, Vstd, speed_mean, tke_like = mean_std(U, V)
    U0 = 60.0 * REGIMES[demo_regime]
    turbulence_intensity = np.sqrt(2/3 * tke_like) / U0 # normalized by U0

    fig, axs = plt.subplots(1, 3, figsize=(15, 4))
    for ax, field, title, cmap in [
        (axs[0], Umean, 'mean u (mm/s)', 'RdBu_r'),
        (axs[1], turbulence_intensity*100, 'turbulence intensity (%)', 'viridis'),
        (axs[2], speed_mean, 'mean speed |U| (mm/s)', 'viridis'),
    ]:
        pc = ax.pcolormesh(X, Y, field, shading='auto', cmap=cmap)
        plt.colorbar(pc, ax=ax)
        ax.set_title(f'{demo_flow} / {demo_regime}: {title}')
        ax.set_xlabel('x (mm)'); ax.set_ylabel('y (mm)')
    plt.tight_layout()
    plt.show()

```



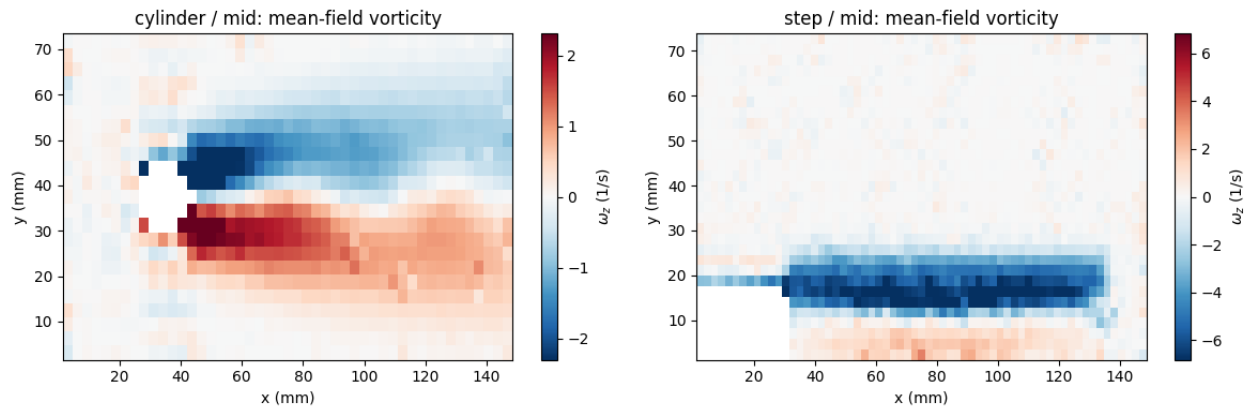
Notice the turbulence intensity peaking exactly on the flanks of the mean wake deficit — the signature of shear-driven fluctuations (here, mostly the sampling of different shedding phases across realizations) rather than measurement noise.

2. Vorticity

The out-of-plane vorticity $\omega_z = \partial v / \partial x - \partial u / \partial y$ is computed from the mean field with a simple centered finite difference (`numpy.gradient`), exactly the kind of derived quantity requested in a PIV lab report.

```
def vorticity(X, Y, U, V):
    dx = X[0, 1] - X[0, 0]
    dy = Y[1, 0] - Y[0, 0]
    dVdx = np.gradient(V, dx, axis=1)
    dUdy = np.gradient(U, dy, axis=0)
    return dVdx - dUdy

to_plot = [flow for flow in ['cylinder', 'step'] if load_batch(flow, 'mid') is not None]
if not to_plot:
    print("Neither cylinder/mid nor step/mid is available yet.")
else:
    fig, axs = plt.subplots(1, len(to_plot), figsize=(6 * len(to_plot), 4), squeeze=False)
    for ax, flow in zip(axs[0], to_plot):
        Xf, Yf, Uf, Vf = load_batch(flow, 'mid')
        Um, Vm, *_ = mean_std(Uf, Vf)
        wz = vorticity(Xf, Yf, Um, Vm)
        vmax = np.nanpercentile(np.abs(wz), 98)
        pc = ax.pcolormesh(Xf, Yf, wz, shading='auto', cmap='RdBu_r',
                           vmin=-vmax, vmax=vmax)
        plt.colorbar(pc, ax=ax, label='$\omega_z$ (1/s)')
        ax.set_title(f'{flow} / mid: mean-field vorticity')
        ax.set_xlabel('x (mm)'); ax.set_ylabel('y (mm)')
    plt.tight_layout()
    plt.show()
```



3. Case-specific diagnostics

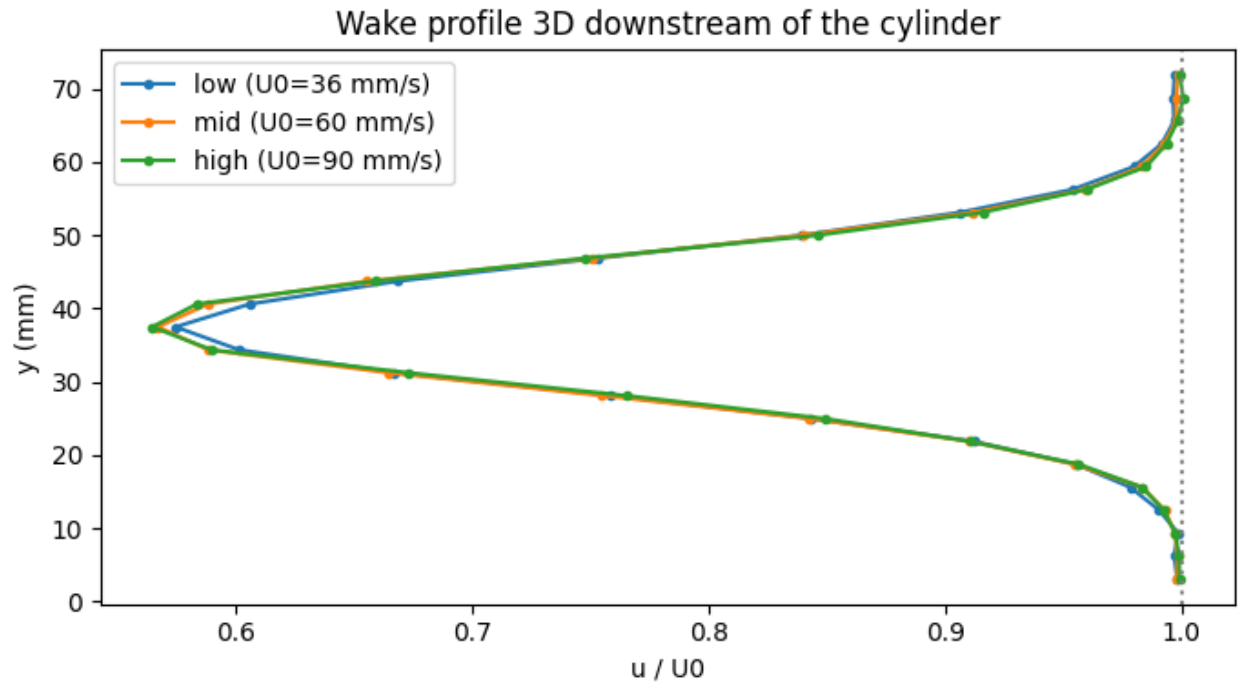
3.1 Cylinder: wake velocity-deficit profile

We extract a cross-stream profile of the mean streamwise velocity a few diameters downstream and read off the deficit relative to the free-stream value — a standard way to characterize a wake.

```
fig, ax = plt.subplots(figsize=(7, 4))
for regime in REGIMES:
    batch = load_batch('cylinder', regime)
    if batch is None:
        print(f"cylinder / {regime}: not processed yet -- skipping")
        continue
    Xc, Yc, Uc, Vc = batch
    Um, *_ = mean_std(Uc, Vc)
    x_probe = CYL_X + 3 * CYL_D
    icol = np.argmin(np.abs(Xc[0, :] - x_probe))
    profile = Um[:, icol]
    y_profile = Yc[:, icol]
    U0 = 60.0 * REGIMES[regime]
    ax.plot(profile / U0, y_profile, marker='o', ms=3, label=f'{regime} (U0={U0:.0f} mm/s)')
    deficit = 1 - np.nanmin(profile) / U0
    print(f"cylinder / {regime}: deficit at x=3D -> {deficit*100:.1f} % of U0")
ax.axvline(1.0, color='gray', ls=':')
ax.set_xlabel('u / U0'); ax.set_ylabel('y (mm)')
ax.set_title('Wake profile 3D downstream of the cylinder')
ax.legend()
plt.tight_layout()
plt.show()
```

cylinder / low: deficit at x=3D -> 42.5 % of U0

cylinder / mid: deficit at $x=3D \rightarrow 43.3\%$ of U_0
cylinder / high: deficit at $x=3D \rightarrow 43.5\%$ of U_0



3.2 Step: recirculation length

Along a row close to the wall, the recirculation bubble shows up as negative mean streamwise velocity; the reattachment point is where it crosses back to positive. We compare the estimate to the length used to generate the synthetic data (STEP_LR_MM) — a validation you will *not* be able to do on real data, but which confirms the method is sound before you trust it there.

```
fig, ax = plt.subplots(figsize=(8, 4))
for regime in REGIMES:
    batch = load_batch('step', regime)
    if batch is None:
        print(f"step / {regime}: not processed yet -- skipping")
        continue
    Xs, Ys, Us, Vs = batch
    Um, *_ = mean_std(Us, Vs)
    irow = np.argmin(np.abs(Ys[:, 0] - 0.4 * STEP_H)) # a row inside the bubble
    u_wall = Um[irow, :]
    x_wall = Xs[irow, :]
    ax.plot(x_wall, u_wall, marker='.', label=regime)

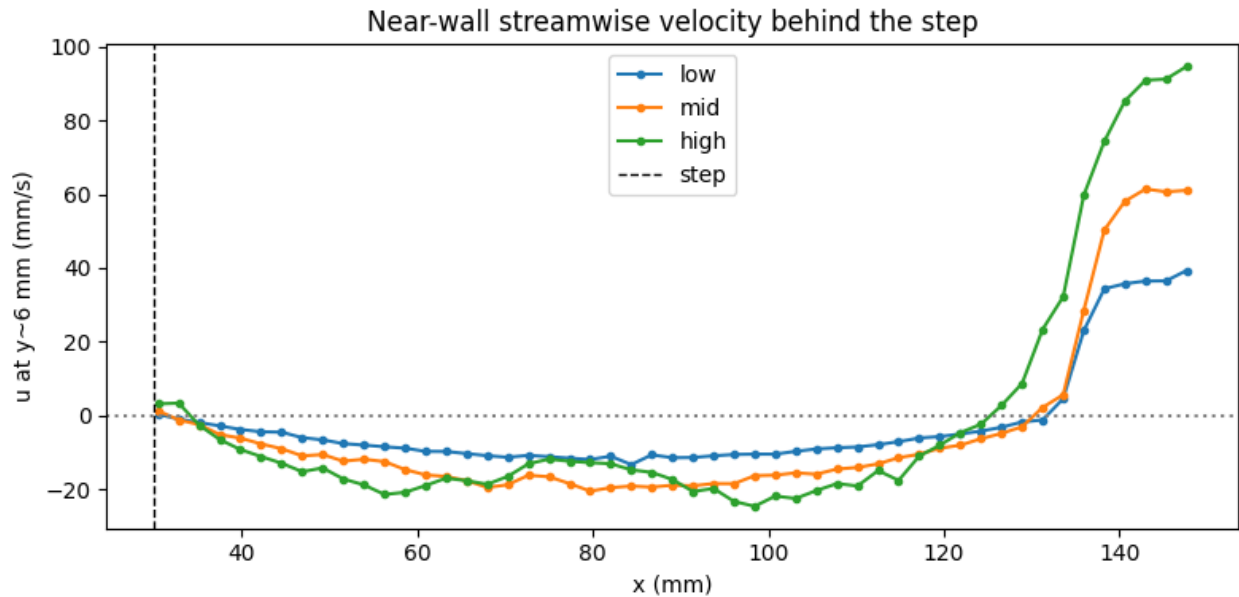
    downstream = x_wall > STEP_X
```

```

sign_change = np.where(np.diff(np.sign(u_wall[downstream]))) > 0)[0]
if len(sign_change):
    x_reattach = x_wall[downstream][sign_change[0]]
    design_str = f", design value = {STEP_LR:.0f} mm" if STEP_LR is not None else ""
    print(f"step / {regime}: estimated reattachment x ~= {x_reattach:.0f} mm "
          f"(Lr ~= {x_reattach - STEP_X:.0f} mm{design_str})")
else:
    print(f"step / {regime}: no reattachment point found -- the near-wall "
          f"recirculation was not resolved (see the invalid-vector "
          f"fraction for this case in notebook 5, section 7)")
ax.axhline(0, color='gray', ls=':')
ax.axvline(STEP_X, color='k', ls='--', lw=1, label='step')
ax.set_xlabel('x (mm)'); ax.set_ylabel(f'u at y~{0.4*STEP_H:.0f} mm (mm/s)')
ax.set_title('Near-wall streamwise velocity behind the step')
ax.legend()
plt.tight_layout()
plt.show()

```

step / low: estimated reattachment x ~= 131 mm (Lr ~= 101 mm, design value = 105 mm)
 step / mid: estimated reattachment x ~= 129 mm (Lr ~= 99 mm, design value = 105 mm)
 step / high: estimated reattachment x ~= 124 mm (Lr ~= 94 mm, design value = 105 mm)



The **step/high** regime originally failed to resolve the recirculation at all (no sign crossing found): heavier reversed-flow gradients and more invalid vectors near the wall (see notebook 5, §7) let the outlier-replacement/smoothing step bias the near-wall

velocity back towards the free-stream value instead of preserving the reversal. Notebook 5, §5 now uses a finer final interrogation window for the `step` flow specifically (48→24→12 px instead of the default 64→32→16 px) to better resolve the thin near-wall shear layer, and the recirculation is recovered at every regime — but note the measured length below is still comparatively less accurate at `high` than at `low/mid`, and §7's invalid-vector fraction is higher everywhere for `step` as a result of the finer window. Window-size choice near sharp gradients is a real resolution/noise trade-off, not just a knob to turn up universally.

4. Vortex shedding & Strouhal number

We probe the transverse velocity a few diameters downstream, look at its spectrum, and convert the dominant frequency into a Strouhal number $St = f_{shed}D/U_0$.

```
def strouhal_from_timeresolved(regime):
    folder = RESULTS_ROOT / 'cylinder' / regime / 'OpenPIV_results_16_timeresolved'
    files = sorted(folder.glob('field_A*.txt'))
    if not files:
        return None
    x0 = y0 = None
    vt = []
    for f in files:
        data = np.loadtxt(f, skiprows=1)
        x, y, u, v, flags, mask = data.T
        if x0 is None:
            x0, y0 = x, y
            target_x = CYL_X + 3 * CYL_D
            target_y = CYL_Y if CYL_Y is not None else y0.mean()
            idx = np.argmin((x0 - target_x)**2 + (y0 - target_y)**2)
            vt.append(v[idx])
    vt = np.array(vt)

    if DATA_SOURCE == "synthetic":
        # dt_frame used during acquisition: recovered from shedding_frequency()
        # the same way notebook 5 built it (6 periods x 16 samples/period)
        f_target = shedding_frequency(regime)
        dt_frame = (1.0 / f_target) / 16
    else:
        f_target = None # unknown for real data -- that's what we're measuring
        dt_frame = DT_FRAME_REAL[regime]
        if dt_frame is None:
            raise ValueError(f"DT_FRAME_REAL['{regime}'] is not set -- match notebook 5 §1.")

    vt_detrend = vt - vt.mean()
```

```

    freqs = np.fft.rfftfreq(len(vt_detrend), d=dt_frame)
    spec = np.abs(np.fft.rfft(vt_detrend))
    f_peak = freqs[1:][np.argmax(spec[1:])]
    return vt, dt_frame, freqs, spec, f_peak, f_target

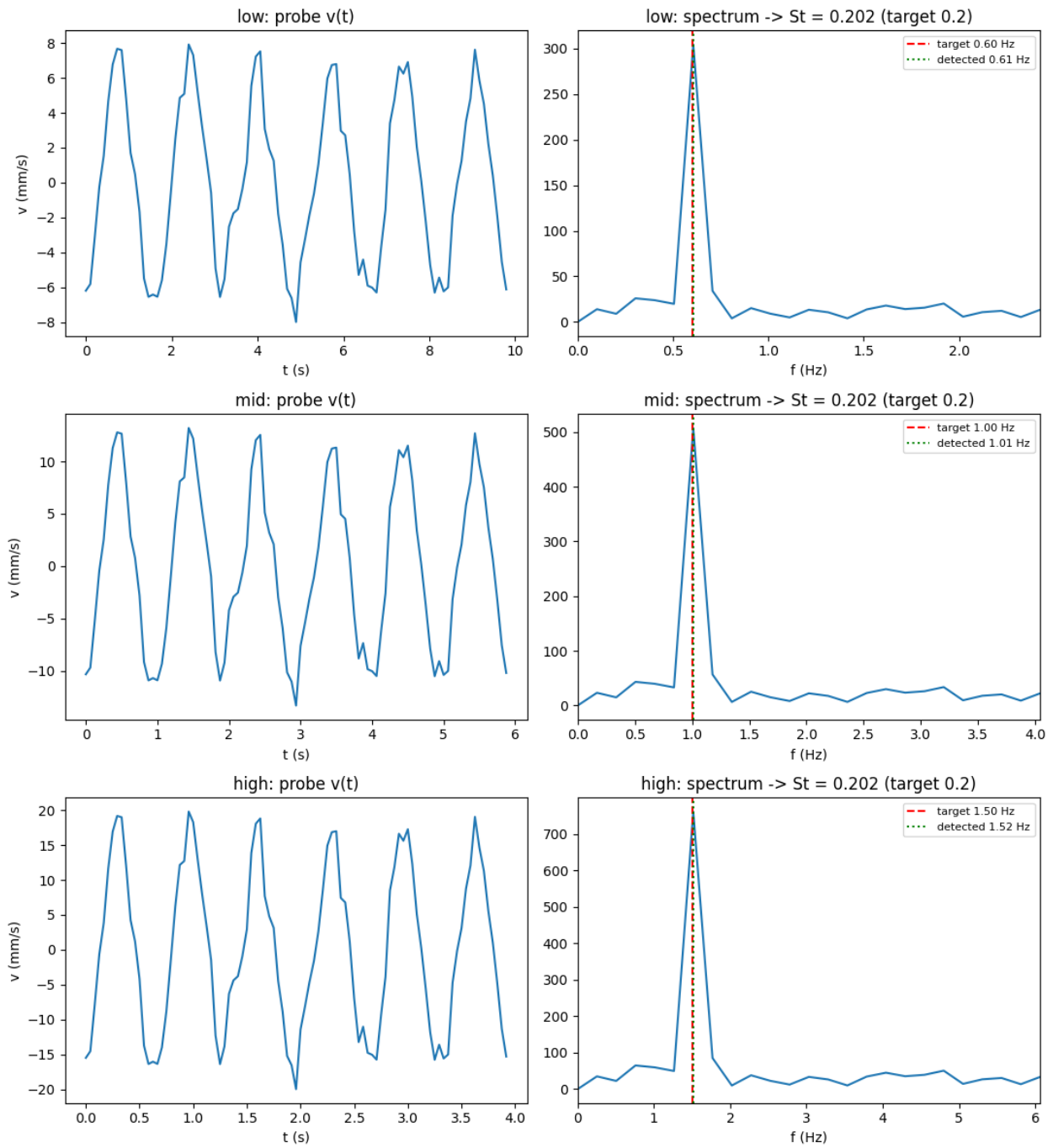
available_regimes = [r for r in REGIMES if strouhal_from_timeresolved(r) is not None]
if not available_regimes:
    print("No time-resolved cylinder sequence processed yet -- run notebook 5 §6 first.")
else:
    fig, axs = plt.subplots(len(available_regimes), 2, figsize=(11, 4 * len(available_regimes)))
    St_values = {}
    for i, regime in enumerate(available_regimes):
        vt, dt_frame, freqs, spec, f_peak, f_target = strouhal_from_timeresolved(regime)
        t = np.arange(len(vt)) * dt_frame
        U0 = 60.0 * REGIMES[regime]
        St = f_peak * CYL_D / U0
        St_values[regime] = St

        axs[i, 0].plot(t, vt)
        axs[i, 0].set_title(f'{regime}: probe v(t)')
        axs[i, 0].set_xlabel('t (s)'); axs[i, 0].set_ylabel('v (mm/s)')

        axs[i, 1].plot(freqs, spec)
        if f_target is not None:
            axs[i, 1].axvline(f_target, color='r', ls='--', label=f'target {f_target:.2f} Hz')
            axs[i, 1].axvline(f_peak, color='g', ls=':', label=f'detected {f_peak:.2f} Hz')
            axs[i, 1].set_xlim(0, max(4 * f_peak, 4 * (f_target or f_peak)))
            st_target_str = f" (target {ST_TARGET})" if DATA_SOURCE == "synthetic" else ""
            axs[i, 1].set_title(f'{regime}: spectrum -> St = {St:.3f}{st_target_str}')
            axs[i, 1].set_xlabel('f (Hz)')
            axs[i, 1].legend(fontsize=8)
    plt.tight_layout()
    plt.show()

    print(f"\n{'regime':6s} {'St (measured)':>15s}" + (" St (target)" if DATA_SOURCE == "synth
    for regime, St in St_values.items():
        line = f"{regime:6s} {St:15.3f}"
        if DATA_SOURCE == "synthetic":
            line += f" {ST_TARGET:13.3f}"
        print(line)

```



regime	St (measured)	St (target)
low	0.202	0.200
mid	0.202	0.200
high	0.202	0.200

The Strouhal number comes out close to the design value at every regime — the classic experimental

observation that, over a wide range of moderate Reynolds numbers, St stays close to 0.2 for a circular cylinder. On real data, expect more scatter (finite sampling, 3D effects, background turbulence) and use this as a sanity check on your probe location and window length rather than an exact target.

5. Regime & flow comparison

Bringing the pieces together: Reynolds number, wake deficit and turbulence intensity across the three flows and three regimes.

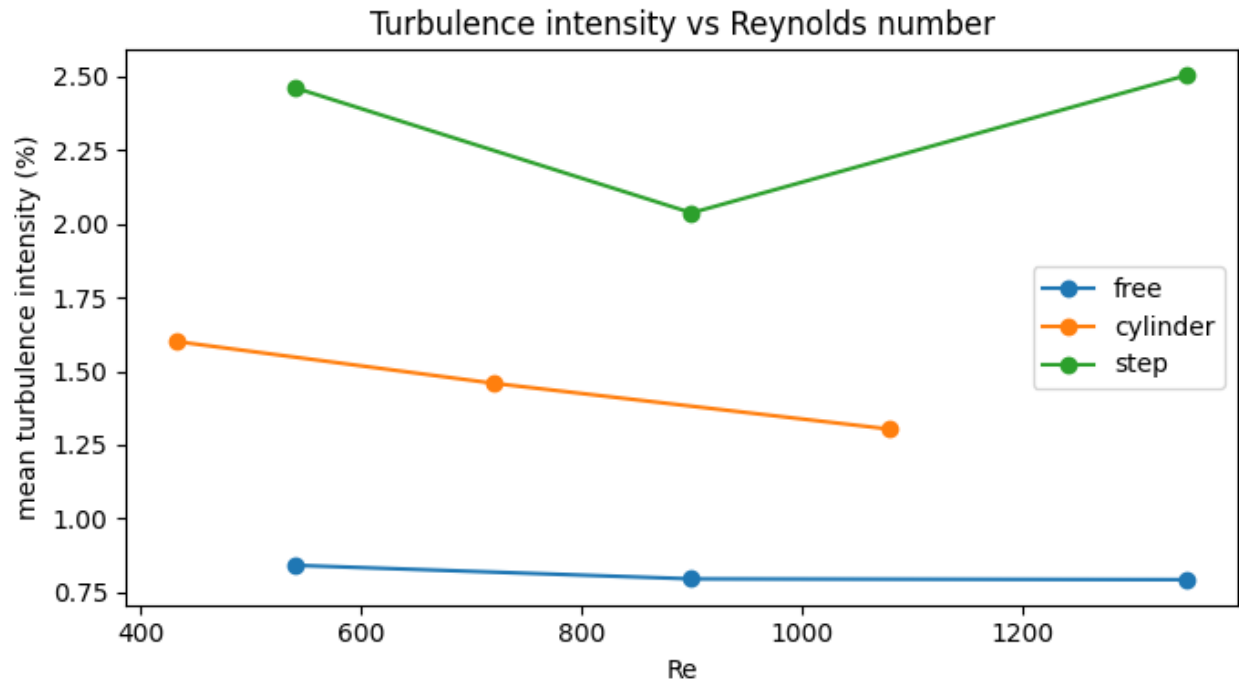
```
rows = []
for flow in FLOWS:
    for regime in REGIMES:
        batch = load_batch(flow, regime)
        if batch is None:
            print(f"{flow} / {regime}: not processed yet -- skipping")
            continue
        Xc, Yc, Uc, Vc = batch
        Um, Vm, Us, Vsd, speed_m, tke = mean_std(Uc, Vc)
        U0 = 60.0 * REGIMES[regime]
        TI = float(np.nanmean(np.sqrt(2/3 * tke)) / U0)
        Re = reynolds_number(flow, regime)
        rows.append((flow, regime, U0, Re, TI))

if not rows:
    print("No processed cases found yet -- run notebook 5 first.")
else:
    print(f"{'flow':10s} {'regime':6s} {'U0(mm/s)':>9s} {'Re':>7s} {'mean TI':>9s}")
    for flow, regime, U0, Re, TI in rows:
        print(f"{'flow':10s} {'regime':6s} {'U0:9.1f} {'Re:7.0f} {'TI*100:8.1f}%")

    fig, ax = plt.subplots(figsize=(7, 4))
    for flow in FLOWS:
        Re_vals = [r[3] for r in rows if r[0] == flow]
        TI_vals = [r[4]*100 for r in rows if r[0] == flow]
        if not Re_vals:
            continue
        ax.plot(Re_vals, TI_vals, marker='o', label=flow)
    ax.set_xlabel('Re'); ax.set_ylabel('mean turbulence intensity (%)')
    ax.set_title('Turbulence intensity vs Reynolds number')
    ax.legend()
    plt.tight_layout()
    plt.show()
```

flow	regime	U0(mm/s)	Re	mean TI
------	--------	----------	----	---------

free	low	36.0	540	0.8%
free	mid	60.0	900	0.8%
free	high	90.0	1350	0.8%
cylinder	low	36.0	432	1.6%
cylinder	mid	60.0	720	1.5%
cylinder	high	90.0	1080	1.3%
step	low	36.0	540	2.5%
step	mid	60.0	900	2.0%
step	high	90.0	1350	2.5%



6. Discussion — for your report

- How does the wake deficit and the recirculation length change with regime? Is that consistent with your expectation for increasing Reynolds number?
- The turbulence intensity estimate mixes true unsteadiness with the realization-to-realization sampling of different shedding phases — how would you separate the two on **time-resolved** data instead of independent realizations?
- The Strouhal number was essentially constant across regimes here — is that still true on your real data? What would make it vary (blockage ratio, free-surface effects, 3D end effects in the aquarium)?
- Compare the three flows at the same regime: how does the free-stream case help you separate “true” flow features from PIV processing artifacts (window size, validation thresholds) in the cylinder and step cases?

- Revisit the image-quality diagnostics: for your real images, do particle density/size vary across flows or regimes to bias these comparisons?

References

- Adrian, R. J. 1991. “Particle-Imaging Techniques for Experimental Fluid Mechanics.” *Annual Review of Fluid Mechanics* 23 (1): 261–304. <https://doi.org/10.1146/annurev.fl.23.010191.000245>.
- Huang, Dabiri, H. 1997. “On Errors of Digital Particle Image Velocimetry.” *Measurement Science and Technology* 8 (12): 1427–39. <https://doi.org/10.1088/0957-0233/8/12/1427>.
- Raffel, Willert, M. 2007. *Particle Image Velocimetry: A Practical Guide*. springer.
- Rougier, Michael Droettboom, Nicolas P. 2014. “Ten Simple Rules for Better Figures.” *PLOS Computational Biology* 10 (9). <https://doi.org/10.1371/journal.pcbi.1003833>.
- Rougier, Nicolas P. 2021. *Scientific Visualization: Python + Matplotlib*. CRC Press.